

Statistiques et Data Science

pour l'analyse de données sportives

Manuel de formation méthodologique
en vue du NFL Big Data Bowl 2027

Corrélations, distributions, tests, régression, EDA,
données de tracking et Machine Learning

Portée. Ce manuel enseigne des *méthodes générales*. Tous les jeux de données utilisés sont **synthétiques ou illustratifs** : ils ne sont pas issus de la compétition et aucune conclusion sur les données réelles de la NFL n'y est affirmée. Aucune sortie d'exécution n'est inventée : les nombres fournis sont soit calculés par les scripts indiqués, soit explicitement marqués comme illustratifs.

Table des matières

Comment utiliser ce manuel	1
I Fondamentaux statistiques	3
1 Comprendre les données	4
1.1 Population, échantillon, observation	4
1.2 Types de variables	4
1.3 Structure temporelle et hiérarchique	4
1.4 Rôle des variables	5
1.5 Association, prédiction, causalité	5
1.6 Pourquoi la structure change la méthode	5
2 Statistiques descriptives	7
2.1 Mesures de tendance centrale	7
2.2 Dispersion	8
2.3 Dispersion relative, forme de la distribution	9
2.4 Fréquences, proportions et tableaux croisés	9
2.5 Statistiques par groupe	9
2.6 Pourquoi la moyenne trompe	10
3 Visualisation statistique	11
3.1 Choisir un graphique selon la question	11
3.2 Les graphiques un à un	11
3.2.1 Histogramme et densité	11
3.2.2 ECDF	11
3.2.3 Boxplot et violon	11
3.2.4 Nuage de points	12
3.2.5 Heatmap et pairplot	12
3.2.6 Résidus, séries temporelles, trajectoires, intervalles de confiance	12
II Corrélations et étude des relations	13
4 Corrélation de Pearson	14
4.1 Problème concret et intuition	14
4.2 Théorie	14
4.3 Code Python	15
4.4 Limites et pièges	15
5 Corrélation de Spearman	17
5.1 Problème et intuition	17
6 Corrélation de Kendall	19
6.1 Idée	19
7 Comparer Pearson, Spearman et Kendall	20
7.1 Confusion par un groupe (paradoxe de Simpson)	20

8	Corrélation, significativité et incertitude	22
8.1	Le raisonnement du test	22
8.2	Rôle de la taille d'échantillon	22
8.3	Absence de preuve $\neq$ preuve d'absence	23
8.4	Comparaisons multiples	23
8.5	Petits et grands échantillons	23
9	Variables catégorielles et numériques	24
9.1	Tableau croisé et chi carré d'indépendance	24
9.2	V de Cramér	24
9.3	Corrélation point-bisériale	25
10	Relations non linéaires et dépendances complexes	26
10.1	Information mutuelle	26
III	Distributions, normalité et valeurs aberrantes	28
11	Comprendre les distributions	29
11.1	Variable individuelle versus moyenne d'échantillon	30
11.2	Effet de la taille d'échantillon	30
12	Étudier la normalité	31
12.1	Pourquoi, et pourquoi pas	31
12.2	Outils	31
12.3	Démarche recommandée	32
13	Valeurs aberrantes et observations influentes	33
13.1	Erreur ou extrême légitime ?	33
13.2	Méthodes de détection	33
13.3	Observations influentes	33
13.4	Winsorisation et transformations	34
14	Transformations et normalisation	35
IV	Tests statistiques et comparaisons	37
15	Construire et tester une hypothèse	38
15.1	De l'observation à la question	38
16	Comparer deux groupes	39
16.1	Test t	39
16.2	Mann–Whitney et Wilcoxon	39
16.3	Tests de permutation	40
17	Comparer plusieurs groupes	41
17.1	ANOVA, Welch, Kruskal–Wallis	41
17.2	Comparaisons post-hoc et correction de Holm	41
18	Chi carré et tests sur les proportions	43
19	Tailles d'effet et intervalles de confiance	44
19.1	Intervalles de confiance par bootstrap	44

V Régression et analyse explicative	45
20 Régression linéaire	46
20.1 Problème et intuition	46
20.2 Qualité d'ajustement et erreurs	46
20.3 Hypothèses et leurs violations	47
21 Régression multiple et variables confondantes	48
21.1 Idée	48
21.2 Variables catégorielles, interactions	48
22 Analyse de résidus et validation	49
22.1 Généralisation	50
23 Corrélation partielle et causalité	51
23.1 Brute, conditionnelle, partielle	51
23.2 Pourquoi l'association ne prouve pas la cause	51
VI Analyse exploratoire avancée (EDA)	52
24 Méthodologie complète d'une EDA	53
25 Qualité des données	54
25.1 Données manquantes	54
25.2 Autres problèmes de qualité	54
26 Jointures et données multi-tables	56
27 Agrégations et analyse par groupe	57
27.1 Choisir l'unité d'analyse	57
28 Séries temporelles et données de tracking	58
28.1 Notions de base	58
28.2 Notions détaillées	58
28.3 Autocorrélation et dépendance	59
VII Statistiques spécifiques au sport et au NFL Big Data Bowl	60
29 Construire de bonnes questions de recherche	61
30 Analyse de performance et comparaisons équitables	62
31 Mesures de tracking et géométrie	63
31.1 Coordonnées et unités	63
31.2 Vitesse, accélération, direction	63
31.3 Distances entre joueurs, voisinage	63
31.4 Bruit, pas de temps, limites	64
32 Éviter les conclusions trompeuses	65

VIII Machine Learning pour compléter l'analyse statistique	66
33 Quand utiliser le Machine Learning ?	67
34 Validation correcte des modèles	68
34.1 Séparation et validation croisée	68
34.2 Fuite de données et pipelines	68
34.3 Séparation adaptée à la structure	69
34.4 Métriques	69
35 Modèles et interprétation	70
36 Réduction de dimension et structures cachées	72
36.1 ACP (PCA)	72
36.2 Clustering	73
IX Python et bibliothèques à maîtriser	74
37 NumPy et Pandas	76
37.1 NumPy	76
37.2 Pandas	76
38 Matplotlib et Seaborn	77
38.1 Matplotlib	77
38.2 Seaborn	77
39 SciPy, statsmodels et Scikit-learn	78
39.1 SciPy (scipy.stats)	78
39.2 statsmodels	78
39.3 Scikit-learn	79
40 Outils d'analyse reproductible	80
A Glossaire statistique	81
B Aide-mémoire : quelle méthode choisir ?	83
C Checklist d'une EDA professionnelle	84
D Références	85

Comment utiliser ce manuel

Ce manuel a un objectif précis : t'apprendre à *raisonner* comme un Data Scientist face à des données sportives, afin que tu puisses, par toi-même, choisir une méthode, la justifier, l'appliquer en Python et défendre tes conclusions. Il ne contient volontairement ni projet de bout en bout, ni exercices, ni dataset de la compétition : tu appliqueras toi-même les méthodes au jeu de données du NFL Big Data Bowl 2027.

Ce que le manuel suppose

Tu connais déjà Python, NumPy, Pandas, Matplotlib, Seaborn et les bases de Scikit-learn. Le manuel ne reprend donc pas Python depuis zéro : il se concentre sur la *statistique*, la *méthodologie* et les *pièges*. Les fonctions Python sont présentées pour ce qu'elles font *statistiquement*.

Structure d'une méthode

Chaque méthode importante est présentée avec le même schéma :

1. la question concrète à laquelle elle répond ;
2. l'intuition ;
3. la théorie (formule, hypothèses) ;
4. une fiche synthétique (objectif, quand l'utiliser, quand *ne pas* l'utiliser, hypothèses, limites, interprétation) ;
5. un exemple simple avec du code Python expliqué ;
6. un exemple lié au sport ou à la NFL, quand c'est pertinent.

Code couleur des encadrés

Intuition

L'idée expliquée avec des mots simples.

Définition

Un terme technique, défini à sa première apparition.

Attention

Un piège ou une limite à ne pas oublier.

Application sportive / NFL

Un exemple inspiré du football américain. Ces exemples sont des *illustrations de raisonnement*, jamais des résultats sur les données réelles.

Honnêteté des exemples

- Les données sont générées avec un générateur aléatoire à graine fixe (`np.random.default_rng(42)`) : tu peux reproduire tous les calculs.
- Quand un résultat numérique est donné dans le texte, il provient d'un calcul explicite ou est signalé comme *illustratif*.
- Les exécutions varient selon les versions de bibliothèques ; vérifie toujours sur ta machine.

Conventions de code. Les extraits supposent les imports suivants :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
rng = np.random.default_rng(42) # generateur reproductible
```

Première partie

Fondamentaux statistiques

Comprendre les données

1.1 Population, échantillon, observation

Vocabulaire de base

- **Population** : l'ensemble complet auquel on veut généraliser (par exemple tous les receveurs ayant joué en NFL).
- **Échantillon** : le sous-ensemble effectivement observé.
- **Observation** (ou individu, ligne) : une unité de données.
- **Unité statistique** : ce que représente une ligne : un joueur, un match, une action, une image de tracking.
- **Variable** : une caractéristique mesurée (colonne).

La première question à se poser devant un tableau est donc : *que représente une ligne ?* Une mauvaise réponse à cette question invalide des analyses entières.

Application sportive / NFL

Une table de tracking a typiquement *une ligne par joueur et par image* (« frame »), alors qu'une table de tests physiques a *une ligne par joueur*. Mélanger ces deux unités sans précaution revient à compter dix mille fois le même joueur.

1.2 Types de variables

Type	Définition	Exemple sportif
Qualitative nominale	catégories sans ordre	poste (WR, CB, QB), équipe
Qualitative ordinale	catégories ordonnées	tour de draft groupé (1 ^{er} , 2 ^e ...), niveau d'expérience
Quantitative discrète	valeurs entières comptées	nombre de plaqués, de passes reçues
Quantitative continue	valeurs réelles mesurées	temps au 40 yards, vitesse, taille

Attention

Un entier n'est pas toujours quantitatif : un identifiant de joueur (`playerId=52341`) ou un numéro de maillot ne se « moyenne » pas. Calculer la moyenne d'un identifiant n'a aucun sens.

1.3 Structure temporelle et hiérarchique

- **Transversales** : plusieurs individus à un instant donné (tous les joueurs d'un Combine).
- **Longitudinales** : les mêmes individus suivis dans le temps.

- **Séries temporelles** : une quantité suivie dans le temps (la vitesse d'un joueur au fil d'une action).
- **Panel** : plusieurs individus suivis plusieurs fois (joueurs $\times$ saisons).

Les données sportives sont presque toujours **hiérarchiques** : des images appartiennent à une action, qui appartient à un match, qui appartient à une saison ; un joueur appartient à une équipe.

Pourquoi la hiérarchie compte

Deux actions du même joueur se ressemblent davantage que deux actions de joueurs différents. Les observations ne sont donc *pas indépendantes*. La plupart des tests classiques supposent l'indépendance ; ignorer la hiérarchie donne des p-values trop optimistes (Chapitres 28 et 34).

1.4 Rôle des variables

Rôle	Description
Variable explicative	ce qu'on utilise pour expliquer ou prédire (vitesse au Combine)
Variable cible	ce qu'on cherche à expliquer ou prédire (une métrique de performance en match)
Identifiant	désigne l'unité, ne porte pas d'information statistique ; sert aux jointures
Variable de confusion	influence à la fois l'explicative et la cible et fabrique une association trompeuse

1.5 Association, prédiction, causalité

Trois questions différentes

- **Association** : deux variables varient-elles ensemble ?
- **Prédiction** : peut-on deviner Y à partir de X sur de nouvelles données ?
- **Causalité** : si l'on *modifiait* X , Y changerait-il ?

Un modèle peut très bien prédire sans qu'aucun effet causal n'existe. Les données observationnelles de la NFL permettent surtout les deux premières ; la troisième exige des hypothèses fortes (Chapitre 23).

Attention

Un joueur plus rapide au Combine joue-t-il mieux *parce qu'il est rapide* ? Ou parce que les joueurs rapides sont sélectionnés pour jouer certains postes, ou entraînés différemment ? La corrélation seule ne tranche pas.

1.6 Pourquoi la structure change la méthode

Le choix d'un test dépend de : le type de variable, le nombre de groupes, l'indépendance ou l'appariement des mesures, la taille de l'échantillon, la forme de la distribution. Avant toute méthode, écris noir sur blanc : *unité d'observation, type de variable, indépendance, taille*. Cette habitude évite la majorité des erreurs.

```
# Exemple : verifier l'unite d'observation d'une table
df = pd.DataFrame({"player_id": [1,1,1,2,2],
                  "frame": [1,2,3,1,2],
                  "speed": [5.1,5.4,5.9,6.2,6.0]})
print(df.shape) # (lignes, colonnes)
print(df["player_id"].nunique()) # nb de joueurs distincts
print(df.groupby("player_id").size()) # nb de lignes par joueur
```

Ligne par ligne : `shape` donne le nombre de lignes et colonnes; `nunique()` compte les valeurs distinctes; `groupby(...).size()` révèle que les joueurs n'ont pas le même nombre de lignes, donc que l'unité n'est *pas* « un joueur ».

Statistiques descriptives

2.1 Mesures de tendance centrale

Moyenne, médiane, mode

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

médiane : valeur qui sépare l'échantillon trié en deux moitiés

mode : valeur la plus fréq

Fiche méthode

Objectif	Résumer la valeur « typique » d'une variable.
Quand l'utiliser	Moyenne : distribution à peu près symétrique, sans extrêmes ; médiane : distribution asymétrique ou avec valeurs extrêmes ; mode : variables catégorielles.
Quand NE PAS l'utiliser	Moyenne : variable très asymétrique ou à queues lourdes sans précaution ; mode : variable continue brute (rarement répétée).
Hypothèses	Moyenne : variable quantitative ; médiane : au moins ordinale.
Limites	La moyenne est tirée par les extrêmes ; la médiane ignore l'information sur l'ampleur des extrêmes ; aucune ne décrit la dispersion.
Interprétation	Si moyenne $\gg$ médiane, la distribution est probablement étirée vers la droite.

```
x = np.array([4.5, 4.6, 4.6, 4.7, 4.8, 4.9, 5.0, 9.9]) # temps (s), un extreme
print(np.mean(x), np.median(x)) # moyenne, mediane
print(stats.mode(x, keepdims=False)) # mode (objet avec .mode et .count)
s = pd.Series(x)
print(s.mean(), s.median(), s.mode().tolist())
```

Calcul à la main : la moyenne vaut $(4,5 + 4,6 + 4,6 + 4,7 + 4,8 + 4,9 + 5,0 + 9,9)/8 = 5,375$, la médiane (moyenne des 4^e et 5^e valeurs) = 4,75. L'unique valeur 9,9 a décalé la moyenne de plus de 0,6 alors que la médiane reste dans la masse des données.

Application sportive / NFL

Les distances parcourues, les gains de yards par action ou les temps de réaction sont souvent asymétriques à droite : quelques actions spectaculaires étirent la queue. Rapporter la *moyenne et la médiane* ensemble est un réflexe sain.

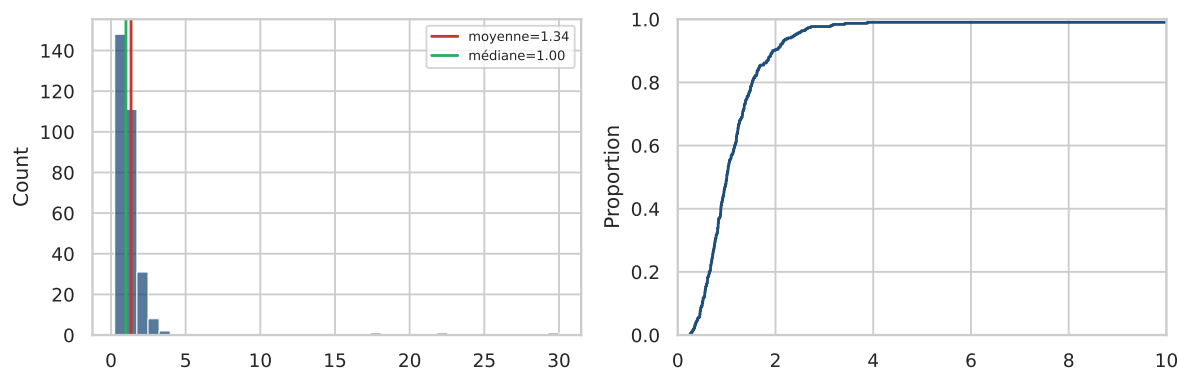


FIG. 2.1 : Données synthétiques log-normales avec trois extrêmes ajoutés. À gauche : la moyenne est tirée vers la droite par rapport à la médiane. À droite : fonction de répartition empirique (ECDF).

2.2 Dispersion

Étendue, variance, écart-type, quantiles

$$R = x_{\max} - x_{\min}, \quad s^2 = \frac{1}{n-1} \sum_i (x_i - \bar{x})^2, \quad s = \sqrt{s^2}.$$

Le **quantile** d'ordre p est la valeur sous laquelle se trouve une proportion p des données ; les **quartiles** sont les quantiles 0,25, 0,5, 0,75 ; l'**IQR** vaut $Q_3 - Q_1$.

Le diviseur $n - 1$ (et non n) corrige un biais : on estime la dispersion autour de $\bar{x}$ et non autour de la vraie moyenne inconnue. En NumPy, `np.var` et `np.std` utilisent par défaut n ; il faut préciser `ddof=1` pour $n - 1$, alors que Pandas utilise $n - 1$ par défaut.

Fiche méthode

Objectif	Quantifier l'étalement des valeurs.
Quand l'utiliser	Écart-type : données quasi symétriques ; IQR : asymétrie, extrêmes ; étendue : contrôle rapide.
Quand NE PAS l'utiliser	Étendue seule pour comparer des groupes de tailles différentes (elle augmente avec n).
Hypothèses	Variance/écart-type : variable quantitative ; IQR : aucune forme imposée.
Limites	Écart-type sensible aux extrêmes ; étendue ne dépend que de deux observations.
Interprétation	Écart-type : écart typique à la moyenne, dans l'unité de la variable ; IQR : largeur des 50 % centraux.

```
print(np.var(x, ddof=1), np.std(x, ddof=1)) # n-1
print(s.var(), s.std()) # Pandas : ddof=1 par défaut
q1, q3 = np.percentile(x, [25, 75]) # quartiles
print(q3 - q1, stats.iqr(x)) # IQR
print(s.quantile([0.1, 0.5, 0.9]))
```

2.3 Dispersion relative, forme de la distribution

Coefficient de variation, asymétrie, aplatissement

$$CV = \frac{s}{\bar{x}} \quad (\bar{x} > 0), \quad \text{skewness} = \frac{1}{n} \sum \left(\frac{x_i - \bar{x}}{s} \right)^3, \quad \text{kurtosis} = \frac{1}{n} \sum \left(\frac{x_i - \bar{x}}{s} \right)^4.$$

Le CV est sans unité : il compare la dispersion *relative* de variables d'échelles différentes (poids en kg, vitesse en m/s). Il n'a de sens que pour une échelle de rapport (zéro naturel). L'asymétrie positive signale une queue à droite ; le kurtosis d'une loi normale vaut 3 (SciPy renvoie par défaut l'*excès* de kurtosis, $\text{kurtosis} - 3$, donc 0 pour une normale).

```
cv = s.std() / s.mean()
print(cv, stats.skew(x), stats.kurtosis(x)) # kurtosis = excès par défaut
print(s.skew(), s.kurt()) # Pandas : estimateurs corrigés
```

Attention

Skewness et kurtosis sont très instables sur de petits échantillons et très sensibles aux extrêmes. Ne les interprète jamais sans graphique.

2.4 Fréquences, proportions et tableaux croisés

```
d = pd.DataFrame({"poste":["WR","WR","CB","CB","CB","QB"],
                  "rapide":[1,0,1,1,0,0]})
print(d["poste"].value_counts()) # effectifs
print(d["poste"].value_counts(normalize=True)) # proportions
print(pd.crosstab(d["poste"], d["rapide"])) # effectifs croisés
print(pd.crosstab(d["poste"], d["rapide"], normalize="index")) # % par ligne
```

`normalize="index"` divise chaque ligne par son total : on lit la *proportion de rapides au sein de chaque poste*. `normalize="columns"` répondrait à une autre question : on lit la *répartition des postes parmi les rapides*. Choisir la bonne normalisation, c'est choisir la bonne question.

2.5 Statistiques par groupe

```
res = pd.DataFrame({"poste": rng.choice(["WR", "CB", "OL"], 300),
                  "temps40": rng.normal(4.7, .25, 300)})
res.loc[res.poste=="OL", "temps40"] += 0.6
print(res.groupby("poste")["temps40"].agg(["count", "mean", "median", "std"]))
sns.boxplot(data=res, x="poste", y="temps40"); plt.show()
```

Attention

Un effectif `count` très petit dans un groupe rend sa moyenne peu fiable. Affiche toujours `count` à côté des moyennes.

2.6 Pourquoi la moyenne trompe

Imagine dix joueurs gagnant chacun 5 yards et un seul gagnant 80 yards : la moyenne vaut environ 12,7 yards, alors qu'aucune action « typique » n'est proche de ce nombre. À l'inverse, supprimer ce 80 supprimerait peut-être *l'événement le plus intéressant*. La règle : décrire par la médiane *et* la moyenne, montrer la distribution, et ne pas supprimer sans raison documentée.

Visualisation statistique

3.1 Choisir un graphique selon la question

Question scientifique	Graphique adapté	Fonction
Quelle est la forme d'une variable ?	histogramme, densité, ECDF	<code>histplot</code> , <code>kdeplot</code> , <code>ecdfplot</code>
Comparer des groupes ?	boxplot, violon, ECDF par groupe	<code>boxplot</code> , <code>violinplot</code>
Lien entre deux variables ?	nuage de points, régression lissée	<code>scatterplot</code> , <code>regplot</code>
Lien entre beaucoup de variables ?	heatmap de corrélations, pairplot	<code>heatmap</code> , <code>pairplot</code>
Évolution dans le temps ?	courbes temporelles	<code>lineplot</code>
Position dans l'espace ?	trajectoires	<code>plt.plot</code> sur (x, y)
Incertitude d'une estimation ?	barres d'intervalles de confiance	<code>pointplot</code> , <code>errorbar</code>

Un graphique est une *réponse à une question*, pas une décoration.

3.2 Les graphiques un à un

3.2.1 Histogramme et densité

Permet de voir : forme, asymétrie, modes multiples. **Ne permet pas de conclure** : sur des valeurs exactes. **Erreurs fréquentes** : choix arbitraire du nombre de classes (**bins**) qui crée ou efface des structures ; courbe de densité (**kdeplot**) lissant trop et débordant au-delà des valeurs possibles. **Bonnes pratiques** : essayer plusieurs **bins**, afficher l'unité.

```
sns.histplot(data=res, x="temps40", bins=30, stat="density", kde=True)
```

3.2.2 ECDF

La fonction de répartition empirique $\hat{F}(t)$ est la proportion de valeurs $\leq t$. Elle ne dépend d'aucun paramètre de lissage et permet de lire des quantiles et de comparer des distributions. Moins intuitive au début, elle est très rigoureuse.

```
sns.ecdfplot(data=res, x="temps40", hue="poste")
```

3.2.3 Boxplot et violon

Le boxplot résume médiane, quartiles et « moustaches » (en général $1,5 \times \text{IQR}$ au-delà des quartiles) et marque les points au-delà. Il *cache* les distributions multimodales et les petits effectifs ; le violon montre la densité mais la lisse. Superposer les points (**stripplot**, **swarmplot**) quand n est petit.

3.2.4 Nuage de points

Montre la forme d'une relation (linéaire, courbe, groupes, extrêmes). Avec beaucoup de points, utiliser la transparence (`alpha`) ou `hexbin/kdeplot` 2D pour éviter la saturation visuelle.

```
sns.scatterplot(data=res, x="temps40", y="temps40", alpha=.4, hue="poste")
```

3.2.5 Heatmap et pairplot

La heatmap d'une matrice de corrélations est un outil de *trriage*, jamais une preuve; elle masque la forme des relations (Chapitre 4). Le pairplot affiche les nuages deux à deux : il complète utilement la matrice mais devient illisible au-delà d'une dizaine de variables.

```
num = pd.DataFrame(rng.normal(size=(200,4)), columns=list("ABCD"))
sns.heatmap(num.corr(), annot=True, vmin=-1, vmax=1, cmap="vlag", center=0)
sns.pairplot(num, corner=True)
```

3.2.6 Résidus, séries temporelles, trajectoires, intervalles de confiance

- **Résidus** : valeurs ajustées en abscisse, résidus en ordonnée; une structure visible révèle un défaut du modèle (Chapitre 22).
- **Séries temporelles** : le temps en abscisse, un joueur ou une action par courbe; attention aux échelles et aux pas de temps irréguliers.
- **Trajectoires** : coordonnées (x, y) ; respecter un repère à échelle égale (`ax.set_aspect("equal")`) sous peine de déformer les distances.
- **Intervalles de confiance** : toujours afficher une estimation *avec* son incertitude; l'absence de barre suggère à tort une précision absolue.

Application sportive / NFL

Pour comparer des postes, un boxplot par poste avec effectifs annotés, ou des ECDF superposées, est plus honnête qu'un diagramme en barres de moyennes.

Attention

Axes tronqués, échelles différentes entre sous-graphiques, couleurs non accessibles, absence d'unités ou de légende : autant de façons de tromper *involontairement*. Chaque figure doit avoir un titre, des axes nommés avec unités et, si besoin, un effectif.

Deuxième partie

Corrélations et étude des relations

Corrélation de Pearson

4.1 Problème concret et intuition

Les joueurs plus rapides au 40 yards ont-ils aussi un meilleur saut vertical ? On veut résumer en un nombre la force et le sens d'une relation *linéaire*.

Intuition

Centre chaque variable autour de sa moyenne. Si les valeurs au-dessus de la moyenne de X tombent surtout avec des valeurs au-dessus de la moyenne de Y (et inversement), les produits $(x_i - \bar{x})(y_i - \bar{y})$ sont majoritairement positifs : la corrélation est positive.

4.2 Théorie

Covariance et corrélation de Pearson

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}), \quad r = \frac{\text{Cov}(X, Y)}{s_X s_Y} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \sqrt{\sum (y_i - \bar{y})^2}}.$$

La covariance dépend des unités ; en la divisant par les deux écarts-types, on obtient $r \in [-1, 1]$, sans unité. Les cas $r = \pm 1$ correspondent à une droite parfaite ; $r \approx 0$ signifie qu'il n'y a pas de relation *linéaire*, pas qu'il n'y a pas de relation. De plus r^2 est la part de variance de Y expliquée par la droite des moindres carrés de Y sur X .

Exemple numérique. Trois points $(1, 2), (2, 4), (3, 5)$: $\bar{x} = 2, \bar{y} = 11/3 \approx 3,667$. Les produits centrés valent $(-1)(-1,667) = 1,667, 0, (1)(1,333) = 1,333$: somme = 3. Les sommes de carrés valent 2 pour x et $2,778 + 0,111 + 1,778 \approx 4,667$ pour y . Donc $r = 3/\sqrt{2 \times 4,667} = 3/3,055 \approx 0,982$ (vérifié avec `stats.pearsonr`).

Fiche méthode

Objectif	Mesurer la force et le sens d'une relation linéaire entre deux variables quantitatives.
Quand l'utiliser	Variables continues, nuage allongé le long d'une droite, peu d'extrêmes.
Quand NE PAS l'utiliser	Relation courbe ou non monotone ; données ordinales ; extrêmes marqués non analysés ; données issues de groupes hétérogènes mélangés.
Hypothèses	Linéarité ; pour l'inférence classique (p-value, intervalle) : observations indépendantes et, idéalement, distribution conjointe approximativement normale.
Limites	Sensible aux valeurs aberrantes ; ne détecte que le linéaire ; ne prouve aucune causalité ; une corrélation faible peut rester pratiquement pertinente, une corrélation forte peut venir d'un groupe ou d'un tiers.
Interprétation	Lire r , le signe, <i>et</i> le nuage de points, <i>et</i> un intervalle de confiance.

4.3 Code Python

```
n = 200
speed = rng.normal(4.7, 0.25, n) # temps 40 yards (s)
jump = 120 - 15*(speed-4.7) + rng.normal(0, 3, n) # saut (cm), relation negative

r, p = stats.pearsonr(speed, jump) # scipy
print(r, p)
ci = stats.pearsonr(speed, jump).confidence_interval(confidence_level=0.95)
print(ci.low, ci.high)

D = pd.DataFrame({"speed": speed, "jump": jump})
print(D.corr(method="pearson")) # matrice (pandas)
print(np.corrcoef(speed, jump)) # matrice 2x2 (numpy)
```

Explication.

- `stats.pearsonr(x, y)` : x et y sont deux vecteurs de même longueur, sans valeurs manquantes. Il renvoie un objet (déconstructible en r , p -value). La p -value teste $H_0 : \rho = 0$ (corrélacion de population nulle).
- `.confidence_interval(confidence_level=0.95)` : intervalle de confiance de ρ , calculé par la transformation de Fisher $z = \operatorname{arctanh}(r)$, d'erreur-type approchée $1/\sqrt{n-3}$, puis ramené par $\tanh$.
- `DataFrame.corr(method="pearson")` : renvoie une matrice carrée de toutes les paires de colonnes numériques; ignore les valeurs manquantes *paire par paire* (le nombre d'observations utilisées varie donc d'une paire à l'autre); l'option `min_periods` fixe un minimum.
- `np.corrcoef(x, y)` : renvoie la matrice 2×2 ; par défaut les variables sont en *lignes* (`rowvar=True`) : avec un tableau d'observations en lignes, il faut `rowvar=False`. Aucune p -value.

Attention

Erreurs fréquentes : passer des tableaux contenant des **NaN** (résultat `nan`), oublier `rowvar=False` avec NumPy, interpréter la p -value comme la *force* de la relation, oublier que la matrice de corrélation est calculée sur des n potentiellement différents.

4.4 Limites et pièges

- **Linéarité** : $y = x^2$ sur un intervalle symétrique a $r \approx 0$ alors que la dépendance est parfaite.
- **Extrêmes** : un seul point éloigné peut créer ou détruire une corrélation (Chapitre 7).
- **Corrélation $\neq$ causalité** : trois explications au moins (X cause Y , Y cause X , un tiers cause les deux), sans compter le hasard.
- **Corrélation faible mais importante** : $r = 0,1$ explique 1 % de la variance mais, sur un phénomène répété des milliers de fois, peut avoir une portée pratique.
- **Matrices** : avec p variables, on calcule $p(p-1)/2$ coefficients; certains seront « significatifs » par hasard (Chapitre 8). Une matrice ne montre ni la forme des relations ni les groupes cachés.

Application sportive / NFL

Une corrélation entre vitesse au Combine et une métrique de performance en match, calculée sur tous les postes mélangés, peut refléter surtout les différences de poste. Calculer *par poste* avant de conclure.

Corrélation de Spearman

5.1 Problème et intuition

Quand la relation est *croissante* mais pas droite (par exemple exponentielle), ou quand les données sont des rangs, Pearson sous-estime la dépendance.

Intuition

Spearman remplace chaque valeur par son **rang** (1 pour la plus petite, etc.), puis calcule Pearson sur les rangs. Il mesure donc la tendance à *monter ensemble*, quelle que soit la forme exacte de la courbe.

Corrélation de Spearman

$$\rho_s = r(\text{rang}(X), \text{rang}(Y)).$$

Sans ex æquo : $\rho_s = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$ avec d_i la différence de rangs. En présence d'ex æquo, on attribue le *rang moyen* (les valeurs 7,7 aux places 3 et 4 reçoivent toutes deux le rang 3,5) et la formule générale (Pearson sur les rangs) est utilisée.

Exemple. $x = (1, 2, 3, 4)$, $y = (1, 4, 9, 16)$: rangs identiques, donc $\rho_s = 1$ alors que Pearson est un peu inférieur à 1 car la courbe n'est pas droite.

Fiche méthode

Objectif	Mesurer une relation monotone (croissante ou décroissante) entre deux variables.
Quand l'utiliser	Relation monotone non linéaire ; présence d'extrêmes ; variables ordinales ; distributions très asymétriques.
Quand NE PAS l'utiliser	Relation non monotone (en U, cloche) ; très petit échantillon avec beaucoup d'ex æquo ; besoin d'une pente (Spearman ne donne pas d'équation).
Hypothèses	Observations indépendantes ; relation monotone.
Limites	Perd l'information sur l'ampleur des écarts ; ne détecte pas les relations non monotones ; ne prouve rien de causal.
Interprétation	$\rho_s = 0,6$: les rangs montent ensemble de façon assez nette ; à lire avec le nuage.

```
x = rng.uniform(0, 4, 150)
y = np.exp(x) + rng.normal(0, 0.5, 150)
print(stats.pearsonr(x, y)[0])
res = stats.spearmanr(x, y)
print(res.statistic, res.pvalue)
print(pd.DataFrame({"x": x, "y": y}).corr(method="spearman"))
```

Explication. `spearmanr(a, b)` accepte deux vecteurs ou une matrice (colonnes = variables, option `axis`) ; l'argument `nan_policy` ("propagate", "omit", "raise") gère les valeurs

manquantes ; **alternative** choisit le test unilatéral ou bilatéral. Le résultat comporte **statistic** et **pvalue**.

Corrélation de Kendall

6.1 Idée

Pour chaque paire d'observations (i, j) , on regarde si elles sont **concordantes** (l'ordre selon X est le même que selon Y) ou **discordantes**.

Tau de Kendall

Avec C paires concordantes, D discordantes, $n_0 = n(n-1)/2$:

$$\tau_a = \frac{C - D}{n_0}, \quad \tau_b = \frac{C - D}{\sqrt{(n_0 - n_1)(n_0 - n_2)}}$$

où n_1 et n_2 comptent les paires ex æquo en X et en Y . SciPy calcule τ_b par défaut (variante qui corrige les ex æquo).

Exemple. Points $(1, 1), (2, 3), (3, 2)$: paires $\{1, 2\}$ concordante, $\{1, 3\}$ concordante, $\{2, 3\}$ discordante. $C = 2$, $D = 1$, $n_0 = 3$ donc $\tau = 1/3$.

Fiche méthode

Objectif

Mesurer l'accord d'ordre entre deux variables via les paires concordantes.

Quand l'utiliser

Petits échantillons, beaucoup d'ex æquo, données ordinales; on veut une interprétation en probabilité.

Quand NE PAS l'utiliser

Très grands échantillons si le temps de calcul compte; relation non monotone.

Hypothèses

Observations indépendantes; relation monotone.

Limites

Coût en $O(n \log n)$ avec les algorithmes modernes mais plus lent que Spearman; valeurs absolues généralement plus petites que ρ_s pour la même relation (on ne les compare donc pas directement).

Interprétation

τ s'interprète comme la différence entre la probabilité d'une paire concordante et celle d'une paire discordante.

```
res = stats.kendalltau(x, y) # tau-b par default
print(res.statistic, res.pvalue)
print(stats.kendalltau(x, y, variant="c").statistic) # variante c (tables rectangulaires)
```

Comparer Pearson, Spearman et Kendall

Cinq situations synthétiques (graine fixe) permettent de voir quand les trois coefficients divergent.

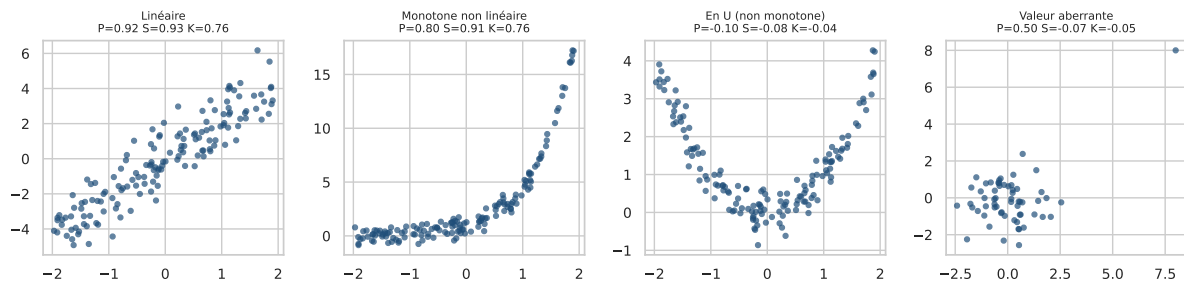


FIG. 7.1 : De gauche à droite : relation linéaire ; monotone non linéaire ; en U ; une valeur aberrante sur un nuage sans relation. P = Pearson, S = Spearman, K = Kendall (valeurs calculées par le script `figs.py`).

Situation	Ce qui se passe	Méthode pertinente	Ce qu'on ne peut pas conclure
Linéaire	les trois élevés et proches	Pearson (donne aussi la pente via régression)	causalité
Monotone linéaire	non Spearman/Kendall plus élevés que Pearson	Spearman ou Kendall	forme précise de la courbe
En U	les trois proches de 0	aucune de ces trois : tracer, information mutuelle, modèle non linéaire	qu'il n'y a pas de relation
Valeur aberrante	Pearson tiré fortement, rangs peu affectés	Spearman/Kendall, ou Pearson avec analyse de l'extrême	que la relation existe pour la masse des données
Confusion groupe	par corrélation globale trompeuse, nulle ou inversée dans chaque groupe	analyser par groupe, ajuster	lien intrinsèque entre X et Y

7.1 Confusion par un groupe (paradoxe de Simpson)

Dans chaque groupe la pente est négative, mais comme les groupes à valeurs de x élevées ont aussi des valeurs de y plus élevées, la corrélation globale est positive.

```
g = np.repeat([0, 1, 2], 60)
xs = rng.normal(3 + 2.5*g, 0.7)
ys = -0.8*xs + 7*g + 2 + rng.normal(0, 0.5, 180)
print("global :", stats.pearsonr(xs, ys)[0])
for k in range(3):
    m = g == k
```

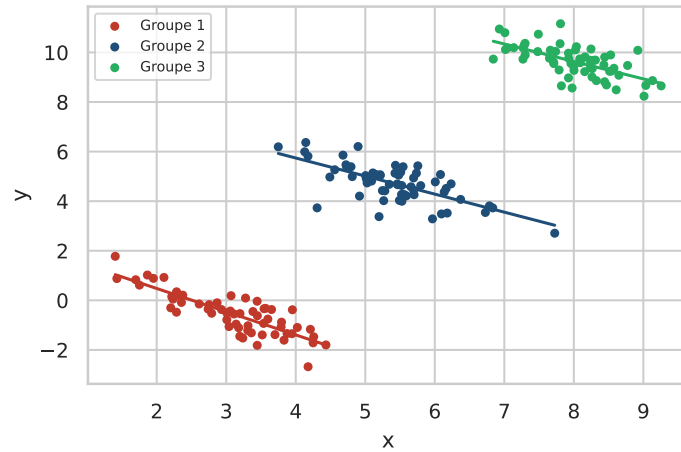


FIG. 7.2 : Trois groupes synthétiques : dans chacun la relation est décroissante, mais la corrélation globale est positive.

```
print("groupe", k, stats.pearsonr(xs[m], ys[m])[0])
```

Attention

Ne présente jamais un coefficient isolé comme preuve. La séquence minimale : *tracer*, calculer plusieurs coefficients, examiner les extrêmes et les sous-groupes, quantifier l'incertitude.

Corrélation, significativité et incertitude

8.1 Le raisonnement du test

Vocabulaire

- **Hypothèse nulle** H_0 : l'hypothèse « pas d'effet » (par exemple $\rho = 0$). **Hypothèse alternative** H_1 : il existe un effet (bilatéral : $\rho \neq 0$).
- **p-value** : probabilité, *si H_0 est vraie*, d'observer un résultat au moins aussi extrême que celui obtenu.
- **Niveau de significativité** α : seuil fixé *avant* l'analyse (souvent 0,05). On rejette H_0 si $p < \alpha$.
- **Intervalle de confiance à 95 %** : procédure qui, répétée sur de nombreux échantillons, contient le vrai paramètre dans environ 95 % des cas.
- **Puissance** $1 - \beta$: probabilité de détecter un effet qui existe réellement.

Attention

Une p-value n'est *pas* la probabilité que H_0 soit vraie, ni celle que le résultat soit dû au hasard, ni une mesure de l'importance de l'effet.

8.2 Rôle de la taille d'échantillon

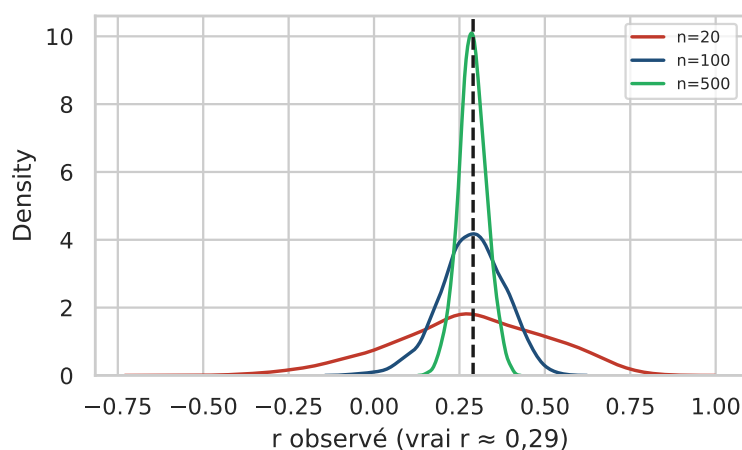


FIG. 8.1 : Distribution simulée du coefficient r observé pour un vrai $\rho \approx 0,29$: plus n est petit, plus r fluctue (la même vraie relation peut sembler nulle ou très forte).

Avec $n = 20$ on peut observer $r = 0,05$ ou $r = 0,6$ pour la même relation ; avec $n = 10\,000$, $r = 0,03$ peut être « significatif » sans aucune importance pratique. Il faut *toujours* rapporter ensemble : r , **intervalle de confiance**, p , n .

```
res = stats.pearsonr(speed, jump)
ci = res.confidence_interval(0.95)
print(f"r={res.statistic:.3f} IC95%=[{ci.low:.3f}; {ci.high:.3f}] p={res.pvalue:.2g} n={len(speed)}")
```

Lecture type (valeurs illustratives). « $r = -0,85$, IC95 % $[-0,88; -0,81]$, $p < 0,001$, $n = 200$ » : corrélation négative forte, estimée avec précision. Un IC large qui contient 0 (« $r = 0,25$, IC $[-0,1; 0,55]$, $n = 20$ ») montre qu'on ne peut *ni* affirmer ni exclure une relation.

8.3 Absence de preuve $\neq$ preuve d'absence

Un résultat non significatif signifie que les données ne permettent pas de rejeter H_0 ; il peut résulter d'un échantillon trop petit (puissance faible). Pour soutenir une absence d'effet pratique, on utilise des intervalles de confiance et des seuils d'équivalence fixés à l'avance.

8.4 Comparaisons multiples

Si l'on teste m relations au seuil 0,05 *sans effet réel*, on attend environ 0,05 m « découvertes » fausses. Pour $m = 100$: environ 5. Corrections :

- **Bonferroni** : seuil α/m (simple, conservatrice).
- **Holm** : plus puissante, même garantie sur le risque d'erreur global (FWER).
- **Benjamini–Hochberg** : contrôle le taux de fausses découvertes (FDR), adaptée à l'exploration.

```
from scipy.stats import false_discovery_control
pvals = np.array([0.001, 0.008, 0.03, 0.04, 0.2, 0.7])
print(false_discovery_control(pvals, method="bh")) # p ajustées BH
m = len(pvals)
holm = np.minimum(1, np.maximum.accumulate((m - np.arange(m)) * np.sort(pvals)))
print(holm) # Holm a la main (p tries croissants)
```

(`statsmodels.stats.multitest.multipletests(p, method="holm")` fait cela directement si `statsmodels` est installé.)

8.5 Petits et grands échantillons

- **Petit n** : estimations instables, puissance faible, extrêmes dominants, hypothèses de normalité difficiles à vérifier.
- **Très grand n** : toute différence minuscule devient « significative » ; les observations de tracking (dépendantes) donnent une *fausse* impression de grand n : le nombre *effectif* d'unités indépendantes est bien plus petit.

Variables catégorielles et numériques

Fiche méthode	
Objectif	Détecter une association entre deux variables catégorielles, ou entre une binaire et une numérique.
Quand l'utiliser	Tableau croisé (chi carré, V de Cramér) ; binaire vs numérique (point-bisérial, comparaison de moyennes/distributions).
Quand NE PAS l'utiliser	Chi carré avec effectifs attendus trop faibles ; point-bisérial avec groupes très déséquilibrés sans précaution.
Hypothèses	Chi carré : observations indépendantes, effectifs attendus ≥ 5 environ dans la grande majorité des cases.
Limites	L'association ne dit ni le sens ni la cause ; la significativité dépend de n .
Interprétation	Toujours associer p-value et taille d'effet.

9.1 Tableau croisé et chi carré d'indépendance

Si les variables sont indépendantes, l'effectif attendu d'une case vaut

$$E_{ij} = \frac{(\text{total ligne}_i)(\text{total colonne}_j)}{n}.$$

La statistique $\chi^2 = \sum \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$ mesure l'écart entre observé O et attendu E .

```
d = pd.DataFrame({"poste": rng.choice(["WR", "CB", "OL"], 400, p=[.4, .4, .2])})
d["rapide"] = (rng.random(400) < np.where(d.poste=="OL", .1, .5)).astype(int)
tab = pd.crosstab(d["poste"], d["rapide"])
chi2, p, dof, expected = stats.chi2_contingency(tab)
print(chi2, p, dof); print(expected)
```

Explication : `chi2_contingency(tab)` prend le tableau d'effectifs (pas les proportions) ; il renvoie la statistique, la p-value, les degrés de liberté $(r - 1)(c - 1)$ et le tableau des effectifs attendus (à vérifier !). Pour les tableaux 2×2 , une correction de continuité de Yates est appliquée par défaut (`correction=True`).

9.2 V de Cramér

$$V = \sqrt{\frac{\chi^2}{n \min(r - 1, c - 1)}} \in [0, 1].$$

Il mesure la *force* de l'association, indépendamment de n (repères indicatifs courants : 0,1 faible, 0,3 moyenne, 0,5 forte pour deux modalités ; à nuancer avec le nombre de modalités).

```
V = np.sqrt(chi2 / (tab.values.sum() * (min(tab.shape) - 1)))
print(V)
```

Alternatives si effectifs insuffisants : regrouper des modalités (avec justification), test exact de Fisher (tableaux 2×2 , Chapitre 18), simulation de Monte-Carlo ou permutation.

9.3 Corrélation point-bisériale

C'est Pearson entre une variable binaire codée 0/1 et une variable numérique : $r_{pb} = \frac{\bar{y}_1 - \bar{y}_0}{s_y} \sqrt{\frac{n_1 n_0}{n^2}}$. Ici s_y est l'écart-type calculé avec le diviseur n . Elle équivaut à un test t à variances égales.

```
y = rng.normal(0, 1, 400) + 0.5*d["rapide"]
r_pb = stats.pointbiserialr(d["rapide"], y)
print(r_pb.statistic, r_pb.pvalue)
```

Attention

Association statistique (V , r_{pb}) et différence de moyennes (Cohen d) sont deux façons de résumer la même idée ; ne les mélange pas dans l'interprétation. Et comparer les distributions complètes (ECDF, boxplots) reste plus informatif que comparer deux moyennes.

Relations non linéaires et dépendances complexes

Pearson, Spearman et Kendall répondent à « les variables croissent-elles ensemble ? ». Beaucoup de dépendances ne sont ni linéaires ni monotones :

- **Quadratique / en U** : un effet qui diminue puis augmente (fatigue vs intensité ?) ;
- **À seuil** : aucun effet jusqu'à une valeur, puis un effet ;
- **Interaction** : l'effet de X_1 dépend de X_2 (la vitesse compte plus pour un poste que pour un autre) ;
- **Dépendance conditionnelle** : X et Y liés seulement une fois Z fixé, ou indépendants une fois Z fixé.

```
x = rng.uniform(-2, 2, 400)
y = x**2 + rng.normal(0, .3, 400)
print(stats.pearsonr(x, y)[0], stats.spearmanr(x, y)[0]) # proches de 0
sns.regplot(x=x, y=y, lowess=True, scatter_kws={"s": 8}) # courbe lissée LOWESS
```

`lowess=True` (nécessite `statsmodels`) trace une courbe locale qui révèle la forme sans imposer de modèle.

10.1 Information mutuelle

Information mutuelle

$I(X;Y) \geq 0$ mesure la réduction d'incertitude sur Y apportée par la connaissance de X . Elle vaut 0 si et seulement si X et Y sont indépendantes, et capte *n'importe quelle* forme de dépendance. Unité : nat (logarithme naturel).

```
from sklearn.feature_selection import mutual_info_regression, mutual_info_classif
X = x.reshape(-1, 1) # forme (n, p) obligatoire
mi = mutual_info_regression(X, y, n_neighbors=3, random_state=0)
print(mi) # un score par colonne de X
# cible binaire :
yb = (y > np.median(y)).astype(int)
print(mutual_info_classif(X, yb, random_state=0))
```

Explication : X doit être 2D ; `n_neighbors` règle le lissage de l'estimateur (par plus proches voisins) ; `discrete_features` indique les colonnes discrètes ; `random_state` fixe le petit bruit ajouté pour la reproductibilité.

Attention

L'information mutuelle ne donne *ni le sens ni la forme* de la relation ; son estimation dépend de n , de `n_neighbors` et du bruit ; il n'existe pas de seuil universel de « forte » MI, et elle n'est pas bornée par 1. Utilise-la comme outil de triage, puis tracer la relation.

Application sportive / NFL

Pour repérer un effet de seuil (par exemple un bénéfice de la vitesse qui n'apparaît qu'au-delà d'une valeur), compare Spearman et information mutuelle : une MI élevée avec un Spearman proche de 0 est un signal pour regarder le nuage.

Troisième partie

Distributions, normalité et valeurs aberrantes

Comprendre les distributions

Vocabulaire

Une **distribution** décrit quelles valeurs une variable prend et avec quelle fréquence. Pour une variable continue, la **densité** $f(x)$ est telle que $P(a \leq X \leq b) = \int_a^b f$; la **fonction de répartition** est $F(x) = P(X \leq x)$. La **distribution empirique** est celle des données observées, avec la fonction de répartition empirique $\hat{F}$.

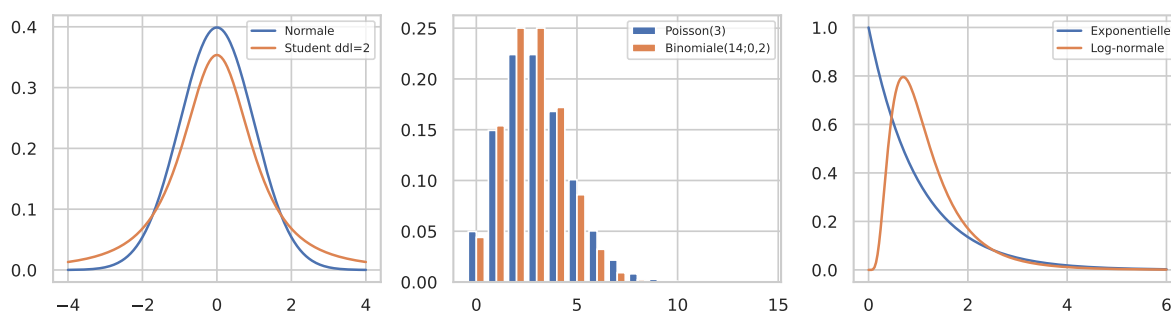


FIG. 11.1 : Quelques lois courantes (paramètres choisis pour l'illustration). Gauche : normale et Student à 2 degrés de liberté (queues lourdes). Centre : Poisson et binomiale. Droite : exponentielle et log-normale.

Loi	Situation typique	Exemple sportif
Normale	somme de nombreux petits effets	erreur de mesure, certaines tailles
Uniforme	toutes les valeurs également possibles	position aléatoire sur un intervalle
Exponentielle	temps d'attente entre événements sans mémoire	délai entre deux actions
Poisson	nombre d'événements dans un intervalle fixe	nombre de passes reçues par match
Binomiale	nombre de succès sur n essais indépendants	complétions sur n passes
Log-normale / asymétriques	produit d'effets, valeurs positives	gains, distances, salaires
Queues lourdes	extrêmes plus fréquents que la normale	performances exceptionnelles

```
print(stats.norm.pdf(0), stats.norm.cdf(1.96)) # densite en 0, P(Z<=1.96)
print(stats.poisson.pmf(3, mu=3), stats.binom.pmf(2, n=10, p=0.2))
sample = stats.expon.rvs(scale=2, size=1000, random_state=0) # echantillon
print(sample.mean(), np.median(sample)) # mean~2, mediane~2 ln2
```

Les objets de `scipy.stats` offrent : `pdf` (ou `pmf` pour les lois discrètes), `cdf` (répartition), `ppf` (quantile, fonction inverse de `cdf`), `rvs` (tirage aléatoire), et `fit` (estimation des paramètres par maximum de vraisemblance).

11.1 Variable individuelle versus moyenne d'échantillon

Intuition

Une variable individuelle peut être très asymétrique. Mais la *moyenne* de n observations indépendantes devient de plus en plus symétrique et proche de la normale quand n augmente (théorème central limite), avec un écart-type $\sigma/\sqrt{n}$ (erreur-type).

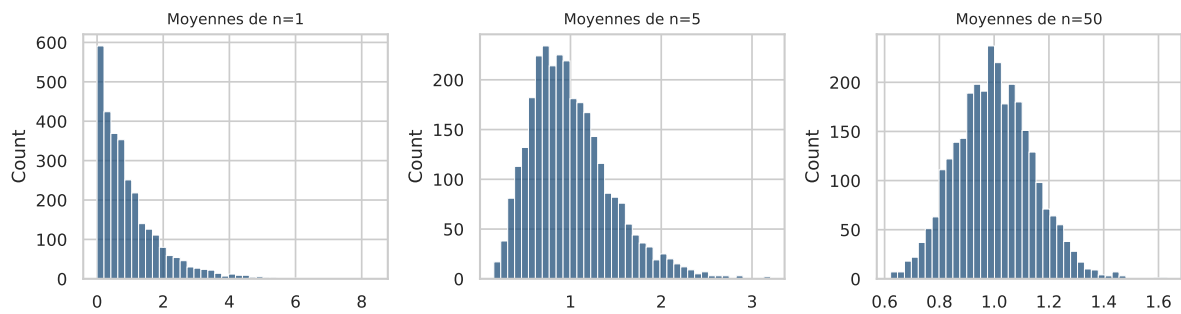


FIG. 11.2 : Moyennes d'échantillons de taille 1, 5 et 50 tirés d'une loi exponentielle (très asymétrique).

Attention

Le théorème central limite demande des observations (à peu près) indépendantes et de variance finie. Avec des queues très lourdes ou des données dépendantes (tracking), la convergence peut être très lente voire trompeuse.

11.2 Effet de la taille d'échantillon

Un petit échantillon peut paraître très irrégulier même s'il provient d'une loi simple. Un histogramme sur 30 points n'autorise que des conclusions prudentes sur la forme.

Étudier la normalité

12.1 Pourquoi, et pourquoi pas

Certaines méthodes (test t , ANOVA, intervalles classiques, régression pour l'inférence) fonctionnent bien quand les *moyennes* ou les *résidus* sont approximativement normaux. Il n'est **pas** nécessaire que toutes les variables le soient : beaucoup de modèles prédictifs (arbres, forêts) n'en ont pas besoin.

12.2 Outils

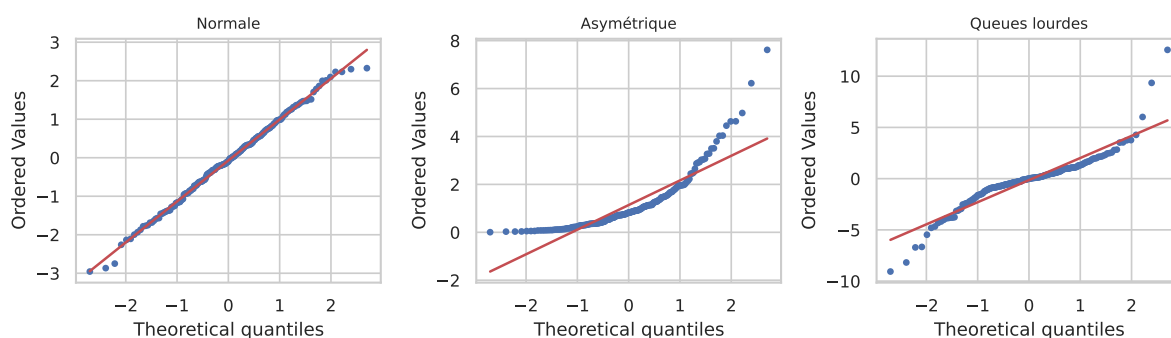


FIG. 12.1 : Q-Q plots synthétiques. Les points suivent la droite pour la loi normale ; une courbure en « J » indique une asymétrie ; des extrémités qui s'écartent des deux côtés indiquent des queues lourdes.

Q-Q plot

Compare les quantiles de l'échantillon aux quantiles théoriques d'une loi normale. Plus les points s'alignent sur la droite, plus la distribution est proche de la normale.

Fiche méthode

Objectif	Évaluer si une variable (ou des résidus) est compatible avec une loi normale.
Quand l'utiliser	Avant un test ou un modèle sensible à la normalité ; pour comprendre la forme des résidus.
Quand NE PAS l'utiliser	Pour « prouver » qu'une variable est normale ; sur de très grands échantillons avec un test (il rejettera pour des écarts négligeables).
Hypothèses	Shapiro–Wilk : observations indépendantes ; D'Agostino–Pearson : $n \geq 8$, combine asymétrie et aplatissement.
Limites	Un test non significatif n'est pas une preuve de normalité ; un test significatif ne dit pas si l'écart importe.
Interprétation	Toujours combiner graphique, ordre de grandeur de l'asymétrie/aplatissement et test.

```

z = rng.exponential(1, 150)
print(stats.shapiro(z)) # statistic, pvalue
print(stats.normaltest(z)) # D'Agostino-Pearson
(osm, osr), (slope, intercept, rr) = stats.probplot(z, dist="norm", plot=None)
stats.probplot(z, dist="norm", plot=plt) # trace le Q-Q plot
# Kolmogorov-Smirnov : attention aux parametres estimates
print(stats.kstest(stats.zscore(z), "norm")) # p-value trop optimiste !

```

Attention

Le test de Kolmogorov–Smirnov classique suppose une loi *entièrement spécifiée* à l'avance. Si la moyenne et l'écart-type sont estimés depuis les mêmes données (comme avec `zscore`), la p-value est trop grande ; on utilise alors la correction de Lilliefors (`statsmodels.stats.diagnostic.lilliefors`).

12.3 Démarche recommandée

1. **Visualiser** (histogramme, densité, Q-Q plot).
2. **Formuler une hypothèse** : *quoi* doit être approximativement normal ? La variable, les résidus, ou la moyenne d'échantillon ?
3. **Utiliser éventuellement un test** (Shapiro–Wilk pour n modéré).
4. **Interpréter la p-value avec prudence** : avec n grand, rejet quasi systématique ; avec n petit, peu de puissance.
5. **Choisir la méthode** : méthode robuste (Welch, Mann–Whitney, permutation, bootstrap) ou transformation si la forme pose réellement problème.

Variables contre résidus

Dans une régression, l'hypothèse de normalité concerne le terme d'*erreur* (estimé par les résidus), pas la variable Y brute. Une variable Y peut être très asymétrique mais produire des résidus corrects si les X expliquent bien cette asymétrie.

Valeurs aberrantes et observations influentes

13.1 Erreur ou extrême légitime ?

Attention

Ne supprime jamais automatiquement une valeur parce qu'elle est éloignée de la moyenne. Dans le sport, une performance extrême peut être l'observation la plus intéressante. On supprime (ou on corrige) uniquement si l'on peut *justifier* que c'est une **erreur** (capteur défaillant, unité mélangée, valeur physiquement impossible), et on documente la décision.

Nature	Indices	Traitement
Erreur de mesure	valeur impossible, saut brutal, unité incohérente	corriger ou exclure <i>avec justification</i>
Extrême légitime	plausible physiquement, cohérent avec le contexte	conserver, analyser séparément, méthodes robustes
Population différente	autre poste, autre contexte	analyser par sous-groupe

13.2 Méthodes de détection

Règles courantes

IQR : une valeur est signalée si $< Q_1 - 1,5 \text{ IQR}$ ou $> Q_3 + 1,5 \text{ IQR}$. **Score** z : $z_i = (x_i - \bar{x})/s$, signalé si $|z| > 3$ environ. **MAD** (écart absolu médian) : $\text{MAD} = \text{median}(|x_i - \text{median}(x)|)$; score z modifié $M_i = 0,6745 (x_i - \text{median})/\text{MAD}$, signalé si $|M_i| > 3,5$ (règle usuelle).

Le score z classique est *lui-même* affecté par l'extrême (il gonfle $\bar{x}$ et s) ; la méthode MAD est plus robuste.

```
a = np.concatenate([rng.normal(10, 1, 100), [18.0]])
q1, q3 = np.percentile(a, [25, 75]); iqr = q3 - q1
flag_iqr = (a < q1 - 1.5*iqr) | (a > q3 + 1.5*iqr)
flag_z = np.abs(stats.zscore(a)) > 3
mad = stats.median_abs_deviation(a, scale="normal") # MAD mis a l'echelle de l'ecart-type
flag_mad = np.abs(a - np.median(a)) / mad > 3.5
print(flag_iqr.sum(), flag_z.sum(), flag_mad.sum())
```

`scale="normal"` multiplie le MAD par 1,4826 pour qu'il estime l'écart-type d'une loi normale, ce qui revient à la constante 0,6745 ci-dessus.

13.3 Observations influentes

Une observation peut être *éloignée* sans être *influyente* (si elle suit la tendance) et inversement, une observation proche du centre en x mais loin de la droite peut peser. En régression, on mesure l'influence par le *levier* et la **distance de Cook** (Chapitre 22). Le test le plus parlant : refaire l'analyse *avec et sans* l'observation et comparer.

13.4 Winsorisation et transformations

La **winsorisation** remplace les valeurs au-delà de percentiles choisis par ces percentiles (par exemple 1% et 99%) au lieu de les supprimer.

```
from scipy.stats.mstats import winsorize
w = winsorize(a, limits=[0.01, 0.01]) # 1 % de chaque cote
print(a.max(), np.asarray(w).max())
```

Les transformations (log, racine) réduisent l'influence des grandes valeurs sans les supprimer (Chapitre 14). **Quand conserver** : extrêmes plausibles, intéressants, ou porteurs de la question de recherche ; dans ce cas, préférer les statistiques robustes (médiane, IQR, Spearman, régression robuste).

Transformations et normalisation

Attention

Deux idées différentes portent le même mot « normaliser » : (i) changer la *forme* d'une distribution pour la rendre plus symétrique ou normale (log, Box–Cox, Yeo–Johnson) ; (ii) changer l'*échelle* des variables pour un algorithme (Z-score, Min-Max, robuste). La première change la distribution ; la seconde ne change que l'unité (et la forme reste la même).

Méthode	Formule / effet	Quand l'utiliser, risques
Z-score	$(x - \bar{x})/s$	KNN, régularisation, ACP, descente de gradient ; la valeur dépend des extrêmes
Min-Max	$(x - \min)/(\max - \min)$	besoin d'un intervalle $[0, 1]$; un extrême écrase tout
Robust scaling	$(x - \text{médiane})/\text{IQR}$	variables avec extrêmes ; échelle moins sensible
Logarithme	$\log(x)$ ou $\log(1 + x)$	données positives, asymétrie droite, effets multiplicatifs ; interprétation en <i>ratios</i>
Racine carrée	$\sqrt{x}$	comptages ; transformation plus douce que le log
Box–Cox	$\frac{x^\lambda - 1}{\lambda}$ ($\lambda = 0 : \log$)	données <i>strictement positives</i> , λ estimé
Yeo–Johnson	variante de Box–Cox	valeurs nulles ou négatives possibles
Normalisation de vecteurs	$x/\ x\ $ par <i>ligne</i>	comparer des profils indépendamment de leur taille (cosinus, texte)

```
from sklearn.preprocessing import (StandardScaler, MinMaxScaler, RobustScaler,
                                   PowerTransformer, Normalizer, FunctionTransformer)
Xc = rng.lognormal(0, 1, (200, 1)) # forme (n, p) obligatoire
Z = StandardScaler().fit_transform(Xc) # moyenne 0, écart-type 1
M = MinMaxScaler().fit_transform(Xc) # entre 0 et 1
R = RobustScaler().fit_transform(Xc) # médiane 0, IQR 1
YJ = PowerTransformer(method="yeo-johnson").fit_transform(Xc)
BC = PowerTransformer(method="box-cox").fit_transform(Xc) # exige x > 0
L = FunctionTransformer(np.log1p).fit_transform(Xc)
print(stats.skew(Xc[:,0]), stats.skew(YJ[:,0]), stats.skew(L[:,0]))
```

Explications. Les transformeurs de Scikit-learn exigent un tableau 2D $((n, p))$. `fit` apprend les paramètres (moyenne, écart-type, λ ...), `transform` les applique ; `fit_transform` fait les deux. **Règle essentielle :** apprendre ces paramètres *uniquement sur l'ensemble d'entraînement*, puis les appliquer à la validation et au test, sinon c'est une fuite de données (Chapitre 34) ; l'outil adapté est `Pipeline`.

Attention

Après transformation logarithmique, une différence de moyennes devient un *ratio* dans l'échelle d'origine ; les médianes se transforment bien, les moyennes non. Standardiser des

variables pour un modèle ne rend *pas* leur distribution normale.

Quatrième partie

Tests statistiques et comparaisons

Construire et tester une hypothèse

15.1 De l'observation à la question

Une observation (« les receveurs semblent plus rapides que les défenseurs ») n'est pas une hypothèse testable. Il faut la transformer en question précise, avec une population, une variable et une comparaison définies.

Étape	Contenu
1. Question	Quelle différence ? Entre quels groupes ? Mesurée comment ?
2. Hypothèses	H_0 : pas de différence (par exemple $\mu_A = \mu_B$); H_1 : différence (bilatérale) ou sens précis (unilatérale, à justifier <i>avant</i> l'analyse)
3. Unité indépendance	et Que représente une ligne ? Les observations sont-elles indépendantes ou appariées ?
4. Méthode	Choisie en fonction de variable, groupes, indépendance, taille, forme
5. Seuils	α fixé <i>avant</i> , taille d'effet d'intérêt minimale, nombre de tests prévus
6. Analyse	Visualiser, tester, quantifier l'effet et son incertitude
7. Conclusion	Limites, alternatives, ce qu'on ne peut pas dire

Attention

Définir la stratégie *avant* de regarder les résultats est ce qui distingue un test d'une exploration déguisée. Tester vingt hypothèses et ne présenter que celle qui passe $p < 0,05$ est le « p-hacking ». Documente ce qui est *confirmatoire* (prévu) et ce qui est *exploratoire* (découvert en chemin).

Application sportive / NFL

Exemple de reformulation : « Les joueurs rapides jouent mieux » devient « Parmi les cornerbacks, la différence de médiane de [métrique de performance] entre le quartile le plus rapide et le quartile le plus lent au 40 yards est-elle non nulle ? » (illustration de formulation, aucun résultat affirmé).

Comparer deux groupes

Fiche méthode	
Objectif	Décider si deux groupes diffèrent sur une variable quantitative, et de combien.
Quand l'utiliser	Deux groupes indépendants (t de Welch, Mann–Whitney, permutation) ou mesures appariées (t apparié, Wilcoxon).
Quand NE PAS l'utiliser	Plus de deux groupes (utiliser ANOVA/Kruskal–Wallis); observations dépendantes traitées comme indépendantes; test seul sans taille d'effet.
Hypothèses	t : indépendance, moyennes approximativement normales (OK si n modéré ou distribution peu asymétrique). Mann–Whitney : indépendance; compare des distributions (pas strictement des médianes sauf formes semblables). Wilcoxon : différences appariées symétriques.
Limites	La p-value ne mesure pas l'importance; les tests rapides ignorent la structure de groupe (joueur, match).
Interprétation	Rapporter différence estimée, IC, taille d'effet et p .

16.1 Test t

Pour deux groupes indépendants, la statistique est $t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}$ (Welch). Le test de Student classique suppose en plus des variances égales; le test de **Welch** ne le suppose pas et est *préférable par défaut*. Le test t **apparié** travaille sur les différences d_i entre deux mesures du même individu : $t = \bar{d}/(s_d/\sqrt{n})$.

```
A = rng.normal(4.60, 0.20, 40) # groupe A
B = rng.normal(4.70, 0.30, 35) # groupe B
print(stats.ttest_ind(A, B, equal_var=False)) # Welch
print(stats.ttest_ind(A, B, equal_var=True)) # Student
avant = rng.normal(5, 1, 30); apres = avant + rng.normal(0.3, 0.5, 30)
print(stats.ttest_rel(avant, apres)) # apparie
```

Explication : `equal_var=False` active Welch; `alternative="two-sided"|"less"|"greater"` choisit le sens de H_1 ; `nan_policy="omit"` ignore les manquants. Le résultat contient `statistic`, `pvalue` et `df` (degrés de liberté, dans les versions récentes de SciPy) ainsi que `confidence_interval()`.

16.2 Mann–Whitney et Wilcoxon

Mann–Whitney (ou Wilcoxon de la somme des rangs) compare deux groupes indépendants via les rangs : il teste si une valeur tirée dans A a une probabilité $\neq 1/2$ d'être supérieure à une valeur tirée dans B. **Wilcoxon signé** est l'équivalent apparié.

```
print(stats.mannwhitneyu(A, B, alternative="two-sided"))
print(stats.wilcoxon(avant, apres))
U = stats.mannwhitneyu(A, B).statistic
```

```
print(U / (len(A)*len(B))) # probabilite de superiorite (effet)
```

16.3 Tests de permutation

Intuition

Si H_0 est vraie (pas de différence), les étiquettes de groupe sont interchangeables. On les mélange de nombreuses fois, on recalcule la statistique à chaque fois, et on regarde la place de la valeur observée dans cette distribution simulée.

```
def diff_med(x, y): return np.median(x) - np.median(y)
perm = stats.permutation_test((A, B), diff_med, permutation_type="independent",
                             n_resamples=9999, random_state=0)
print(perm.statistic, perm.pvalue)
```

Les tests de permutation s'adaptent à *n'importe quelle* statistique (médiane, corrélation...) et n'exigent pas de loi particulière, mais ils supposent l'**échangeabilité** sous H_0 : mélanger des lignes dépendantes (même joueur) viole cette condition ; il faut alors permuter *par groupe de dépendance*.

Attention

Choisir entre test t et Mann–Whitney *après* avoir regardé la p-value est une forme de p-hacking. Décide d'avance, ou rapporte les deux et vérifie qu'ils concordent.

Comparer plusieurs groupes

17.1 ANOVA, Welch, Kruskal–Wallis

Comparer trois groupes ou plus par des tests t successifs multiplie le risque d'erreur. L'**ANOVA à un facteur** teste $H_0 : \mu_1 = \dots = \mu_k$ via le rapport de la variance *entre* groupes à la variance *intra* groupes : $F = \frac{SC_{\text{inter}}/(k-1)}{SC_{\text{intra}}/(n-k)}$. Une p-value faible indique qu'*au moins un* groupe diffère, sans dire lequel.

Fiche méthode

Objectif	Détecter une différence globale entre $k \geq 3$ groupes.
Quand l'utiliser	ANOVA : variances comparables, résidus peu éloignés de la normale ; Welch : variances inégales ; Kruskal–Wallis : distributions asymétriques ou ordinales.
Quand NE PAS l'utiliser	Groupes non indépendants (mesures répétées) ; effectifs minuscules ; interpréter une p-value significative comme « tous différents ».
Hypothèses	Indépendance ; ANOVA : normalité approximative des résidus, homogénéité des variances.
Limites	Teste seulement l'hypothèse globale ; il faut ensuite des comparaisons post-hoc corrigées.
Interprétation	Rapporter F ou H , ddl, p et une taille d'effet (η^2 ou ε^2).

```
g1, g2, g3 = rng.normal(4.6,.2,30), rng.normal(4.7,.2,30), rng.normal(4.9,.2,30)
print(stats.f_oneway(g1, g2, g3)) # ANOVA
print(stats.kruskal(g1, g2, g3)) # Kruskal-Wallis
print(stats.levene(g1, g2, g3)) # egalite des variances
# eta^2 = SC_inter / SC_total
allv = np.concatenate([g1, g2, g3]); gm = allv.mean()
sc_inter = sum(len(g)*(g.mean()-gm)**2 for g in (g1, g2, g3))
sc_tot = ((allv-gm)**2).sum()
print(sc_inter / sc_tot)
```

SciPy ne propose pas directement l'ANOVA de Welch ; avec `statsmodels`, `statsmodels.stats.oneway.anova_oneway(data, use_var="unequal")` la fournit (bibliothèque pingouin également).

17.2 Comparaisons post-hoc et correction de Holm

Après un test global significatif, on compare les groupes deux à deux *avec correction*.

- **Tukey HSD** (`stats.tukey_hsd`) : paires de moyennes après ANOVA.
- **Mann–Whitney par paires + Holm** : alternative non paramétrique (ou test de Dunn).

```
print(stats.tukey_hsd(g1, g2, g3))
from itertools import combinations
paires = list(combinations(range(3), 2)); groupes = [g1, g2, g3]
p = np.array([stats.mannwhitneyu(groupe[i], groupe[j]).pvalue for i, j in paires])
```

```
ordre = np.argsort(p); m = len(p)
holm = np.empty(m)
holm[ordre] = np.minimum(1, np.maximum.accumulate((m - np.arange(m)) * p[ordre]))
print(dict(zip(paires, holm.round(4))))
```

La correction de **Holm** ordonne les p-values de la plus petite à la plus grande, multiplie la i -ème par $m - i + 1$ puis impose la monotonie ; elle domine Bonferroni en puissance avec le même contrôle de l'erreur globale.

Application sportive / NFL

Comparer cinq postes donne dix paires. Sans correction, la probabilité d'au moins une fausse « différence significative » dépasse 40 % (calcul : $1 - 0,95^{10} \approx 0,40$). Annonce toujours *combien* de comparaisons ont été faites.

Chi carré et tests sur les proportions

Fiche méthode

Objectif	Tester l'indépendance de deux variables catégorielles ou comparer des proportions.
Quand l'utiliser	Chi carré : tableaux $r \times c$ avec effectifs suffisants ; Fisher : petits effectifs 2×2 ; z de proportions : deux proportions, grand échantillon.
Quand NE PAS l'utiliser	Chi carré avec effectifs attendus < 5 dans plusieurs cases ; données appariées (utiliser McNemar) ; données dépendantes.
Hypothèses	Indépendance des observations ; effectifs attendus suffisants (règle approximative : tous ≥ 1 et au moins 80 % des cases ≥ 5).
Limites	Significativité dépend de n ; ne donne pas le sens ; n'indique pas quelles cases contribuent (examiner résidus standardisés).
Interprétation	Toujours ajouter une taille d'effet (V de Cramér, différence ou rapport de proportions, odds ratio) et un intervalle de confiance.

Fréquence attendue

Sous l'hypothèse d'indépendance, $E_{ij} = \frac{\text{total ligne}_i \times \text{total colonne}_j}{n}$.

```
tab = np.array([[18, 12], [7, 23]]) # effectifs 2x2
chi2, p, dof, E = stats.chi2_contingency(tab, correction=False)
print(chi2, p, E) # vérifier E
odds, pf = stats.fisher_exact(tab) # test exact (2x2)
print(odds, pf)
res = stats.contingency.association(tab, method="cramer")
print(res) # V de Cramer
```

Comparer deux proportions. Avec $\hat{p}_1 = x_1/n_1$ et $\hat{p}_2 = x_2/n_2$, la différence $\hat{p}_1 - \hat{p}_2$ a pour erreur-type approchée $\sqrt{\hat{p}_1(1 - \hat{p}_1)/n_1 + \hat{p}_2(1 - \hat{p}_2)/n_2}$. Avec `statsmodels : proportions_ztest, proportion_confint` ; sans lui, `stats.binomtest(k, n).proportion_ci()` donne un intervalle (Clopper–Pearson par défaut) pour une proportion.

Attention

Un odds ratio n'est pas un risque relatif : lorsque l'événement est fréquent, il exagère l'effet. Précise toujours lequel tu rapportes.

Tailles d'effet et intervalles de confiance

Une p-value répond à « l'effet est-il compatible avec zéro ? », jamais à « l'effet est-il grand ? ». Il faut donc rapporter la **taille d'effet** et son **incertitude**.

Situation	Taille d'effet	Remarque
Deux moyennes	différence $\bar{x}_1 - \bar{x}_2$ (unité d'origine) ; Cohen $d = \frac{\bar{x}_1 - \bar{x}_2}{s_p}$	repères indicatifs 0,2 / 0,5 / 0,8 (conventionnels, à contextualiser)
Non paramétrique	probabilité de supériorité $U/(n_1 n_2)$	interprétable directement
Deux proportions	différence, risque relatif, odds ratio	préciser la mesure
Association catégorielle	V de Cramér	indépendant de n
Corrélation	r, ρ_s, r^2	voir Chapitre 8
Plusieurs groupes	η^2, ε^2	part de variance

avec l'écart-type combiné $s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$.

```
def cohen_d(a, b):
    n1, n2 = len(a), len(b)
    sp = np.sqrt(((n1-1)*a.var(ddof=1) + (n2-1)*b.var(ddof=1)) / (n1+n2-2))
    return (a.mean() - b.mean()) / sp
print(cohen_d(A, B))
```

19.1 Intervalles de confiance par bootstrap

Intuition

On ne connaît pas la population ; on « rééchantillonne » avec remise l'échantillon observé des milliers de fois, on recalcule la statistique à chaque fois, et les percentiles de ces valeurs donnent un intervalle.

```
def diff_means(x, y, axis=-1): return x.mean(axis=axis) - y.mean(axis=axis)
bs = stats.bootstrap((A, B), diff_means, vectorized=True, n_resamples=9999,
                    confidence_level=0.95, method="BCa", random_state=0)
print(bs.confidence_interval)
```

method="BCa" corrige le biais et l'asymétrie ; "percentile" est plus simple. Le bootstrap suppose des observations *indépendantes* : pour des données groupées (plusieurs lignes par joueur), il faut rééchantillonner les *joueurs*, pas les lignes (bootstrap par blocs/clusters).

Attention

Un effet peut être statistiquement significatif mais négligeable (gain de 0,01 seconde), ou non significatif mais potentiellement important si l'intervalle est large et contient des valeurs cruciales. Pose à l'avance la plus petite différence qui t'intéresserait.

Cinquième partie

Régression et analyse explicative

Régression linéaire

20.1 Problème et intuition

Corréler dit « ça varie ensemble » ; la régression répond à « de *combien* Y varie-t-il quand X augmente d'une unité ? » et permet de prédire.

Modèle de régression linéaire simple

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i.$$

β_0 est l'**intercept** (valeur prédite pour $x = 0$, parfois hors du domaine observé), β_1 la **pente** (variation moyenne de Y pour +1 sur X), ε_i l'erreur. Les moindres carrés ordinaires choisissent $\hat{\beta}_0, \hat{\beta}_1$ minimisant $\sum (y_i - \hat{y}_i)^2$; on obtient $\hat{\beta}_1 = \frac{\text{Cov}(X, Y)}{\text{Var}(X)} = r \frac{s_Y}{s_X}$ et $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$. Les **résidus** sont $e_i = y_i - \hat{y}_i$.

Exemple numérique. Avec les points $(1, 2), (2, 4), (3, 5)$: $\hat{\beta}_1 = \text{Cov} / \text{Var} = 3/2 = 1,5$ (somme des produits centrés = 3, somme des carrés de $x = 2$) et $\hat{\beta}_0 = 3,667 - 1,5 \times 2 = 0,667$.

20.2 Qualité d'ajustement et erreurs

$$R^2 = 1 - \frac{\sum e_i^2}{\sum (y_i - \bar{y})^2}, \quad R_{\text{aj}}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1},$$

$$\text{MAE} = \frac{1}{n} \sum |e_i|, \quad \text{MSE} = \frac{1}{n} \sum e_i^2, \quad \text{RMSE} = \sqrt{\text{MSE}}.$$

R^2 est la part de variance de Y expliquée par le modèle (en régression simple, $R^2 = r^2$) ; il *augmente toujours* quand on ajoute des variables, d'où R_{aj}^2 qui pénalise la complexité. MAE et RMSE s'expriment dans l'unité de Y ; le RMSE pénalise davantage les grosses erreurs.

Fiche méthode

Objectif	Quantifier et prédire l'effet linéaire d'une ou plusieurs variables sur une variable quantitative.
Quand l'utiliser	Relation à peu près linéaire, besoin d'une pente interprétable, comparaison d'effets ajustés.
Quand NE PAS l'utiliser	Cible catégorielle (régression logistique) ; relation fortement courbe sans transformation ; données dépendantes ignorées ; extrapolation loin des données.
Hypothèses	Linéarité ; indépendance des erreurs ; variance des erreurs constante (homoscédasticité) ; pour l'inférence : erreurs approximativement normales ; absence de multicollinéarité forte et de points trop influents.
Limites	Associations, pas causalité ; sensible aux extrêmes ; R^2 élevé $\neq$ bon modèle.
Interprétation	La pente est un effet <i>moyen</i> associé, dans l'unité de Y par unité de X , avec son intervalle de confiance.

```

from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
n = 200
x = rng.uniform(4.4, 5.2, n)
y = 130 - 15*(x - 4.4)*2 + rng.normal(0, 4, n) # saut en fonction du temps
X = x.reshape(-1, 1)
mod = LinearRegression().fit(X, y)
print(mod.intercept_, mod.coef_)
yhat = mod.predict(X)
print(r2_score(y, yhat), mean_absolute_error(y, yhat),
      np.sqrt(mean_squared_error(y, yhat)))

```

Explication : X doit être 2D ; fit estime ; coef_ est la pente, intercept_ l'intercept ; predict calcule les valeurs ajustées. **Scikit-learn ne fournit ni p-values ni intervalles de confiance des coefficients** : c'est le rôle de statsmodels.

```

import statsmodels.api as sm # si installe : pip install statsmodels
Xc = sm.add_constant(x) # ajoute la colonne d'intercept
fit = sm.OLS(y, Xc).fit()
print(fit.summary()) # coefficients, erreurs-types, IC, R2, tests
print(fit.conf_int()) # IC95% des coefficients

```

sm.add_constant est indispensable : OLS n'ajoute pas d'intercept automatiquement. Le résumé donne coefficients, erreurs-types, statistiques t , p-values, IC, R^2 et plusieurs diagnostics.

Attention

Un R^2 élevé ne prouve *ni* causalité *ni* validité générale : il peut résulter d'un sur-ajustement, d'une tendance commune (deux variables croissant dans le temps), d'un groupe confondant. Et un R^2 faible n'invalide pas un effet réel quand le phénomène est intrinsèquement bruité (comme souvent en sport).

20.3 Hypothèses et leurs violations

Hypothèse	Signe de violation	Remède possible
Linéarité	courbure dans le graphe des résidus	transformation, termes quadratiques, splines
Indépendance	erreurs corrélées (temps, joueur)	modèles à effets mixtes, erreurs-types par cluster
Homoscédasticité	résidus en « entonnoir »	transformation de Y, erreurs-types robustes, WLS
Normalité des erreurs	Q-Q plot des résidus déformé	moins critique si n grand ; bootstrap
Multicolinéarité	VIF élevé, coefficients instables	retirer/combiner des variables, régularisation (ridge)
Points influents	grande distance de Cook	vérifier la donnée, analyse avec/sans, régression robuste

La **multicolinéarité** (variables explicatives très corrélées entre elles) ne biaise pas les prédictions mais rend les coefficients individuels instables et difficiles à interpréter ; le facteur d'inflation de la variance $VIF_j = 1/(1 - R_j^2)$ la mesure (valeurs au-delà de 5–10 : alerte conventionnelle).

Régression multiple et variables confondantes

21.1 Idée

$$y_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} + \varepsilon_i.$$

Chaque β_j est un **coefficient conditionnel** : l'effet associé à +1 sur x_j en gardant les autres variables du modèle constantes. L'association brute et l'association ajustée peuvent différer, y compris de signe.

```
n = 400
poste = rng.choice([0, 1], n) # 0 = type A, 1 = type B (ex. poids eleve)
poids = 90 + 25*poste + rng.normal(0, 8, n)
temps = 4.5 + 0.02*(poids-90) + rng.normal(0, .1, n) # le poids ralentit
perf = 50 - 8*(temps-4.5) + 10*poste + rng.normal(0, 3, n)
D = pd.DataFrame({"poste": poste, "poids": poids, "temps": temps, "perf": perf})
from sklearn.linear_model import LinearRegression
print(LinearRegression().fit(D[["temps"]], D.perf).coef_) # brut
print(LinearRegression().fit(D[["temps", "poste"]], D.perf).coef_) # ajuste
```

Ici le type de poste fait monter à la fois le temps (via le poids) et la performance. Le coefficient brut de **temps** mélange donc les deux effets : avec la graine de ce manuel il est *positif* (comme si être plus lent améliorerait la performance), alors que le coefficient ajusté est *négatif* et proche de la vraie valeur -8 utilisée pour simuler (l'écart restant vient du bruit d'échantillonnage). C'est la raison d'être de l'ajustement. **Mais** l'ajustement ne corrige que les confondants *mesurés et correctement spécifiés*.

21.2 Variables catégorielles, interactions

Une variable catégorielle se code en **indicateurs** (*one-hot*), en retirant une modalité de référence pour éviter la colinéarité parfaite.

```
dd = pd.get_dummies(D.assign(postes=D.postes.map({0:"A", 1:"B"})), columns=["postes"], drop_first=True)
D["temps_x_postes"] = D.temps * D.postes # terme d'interaction
```

Une **interaction** $x_1 x_2$ permet à l'effet de x_1 de dépendre de x_2 ; quand elle est dans le modèle, les termes principaux ne s'interprètent plus seuls.

Attention

Interprétation prudente : un coefficient n'est pas « l'effet de x_j » mais l'association conditionnelle, valide seulement si le modèle est bien spécifié. Ajouter des variables « pour être sûr » peut aussi *introduire* un biais (contrôler un collisionneur ou un médiateur, Chapitre 23).

Analyse de résidus et validation

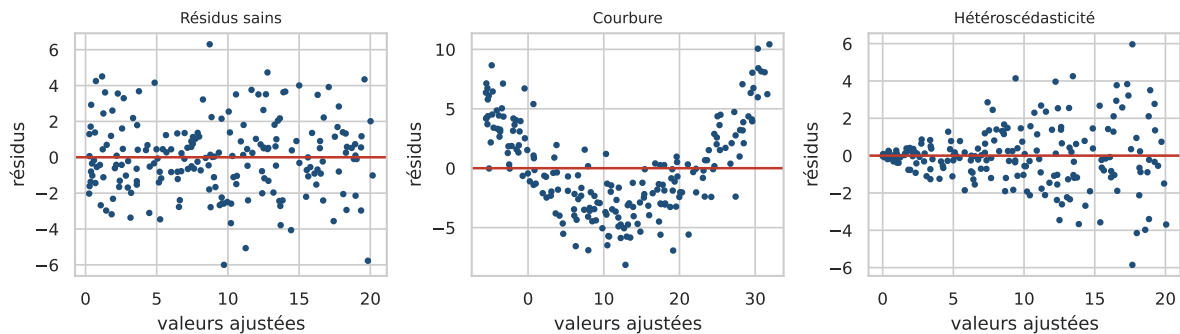


FIG. 22.1 : Résidus en fonction des valeurs ajustées (données synthétiques). Gauche : aucun motif (sain). Centre : courbure (relation non linéaire oubliée). Droite : dispersion croissante (hétéroscédasticité).

Fiche méthode

Objectif

Vérifier que le modèle respecte ses hypothèses et généralise.

Quand l'utiliser

Après tout ajustement linéaire, avant d'interpréter les coefficients.

Quand NE PAS l'utiliser

Se contenter d'un test global (R^2 , p-value) sans regarder les résidus.

Hypothèses

Les résidus doivent être « sans structure » autour de 0.

Limites

Les tests de diagnostic (Breusch-Pagan, Shapiro-Wilk) ont les mêmes limites que tous les tests ; privilégier les graphiques.

Interprétation

Une structure visible = hypothèse violée = conclusions fragiles.

```
res = y - yhat # residus du modele precedent
plt.scatter(yhat, res, s=8); plt.axhline(0, color="r") # linearite, homoscedasticite
stats.probplot(res, dist="norm", plot=plt) # normalite des residus
# si statsmodels est installe
from statsmodels.stats.diagnostic import het_breuschpagan
print(het_breuschpagan(fit.resid, fit.model.exog)) # LM stat, p, F stat, p
infl = fit.get_influence()
cook = infl.cooks_distance[0] # distance de Cook
print(np.argsort(cook)[-3:]) # 3 points les plus influents
```

La **distance de Cook** mesure combien les coefficients changent si l'on retire un point ; une valeur nettement supérieure aux autres (repère courant : $> 4/n$) justifie un examen, *pas une suppression automatique*.

22.1 Généralisation

Un modèle évalué sur les données d'entraînement est trop optimiste. Il faut évaluer sur des données *non vues* (hors échantillon), en respectant la structure des données (Chapitre 34).

Corrélation partielle et causalité

23.1 Brute, conditionnelle, partielle

La corrélation brute r_{XY} ignore les autres variables. La **corrélation partielle** $r_{XY.Z}$ est la corrélation entre X et Y après avoir retiré de chacune la part linéaire expliquée par Z :

$$r_{XY.Z} = \frac{r_{XY} - r_{XZ}r_{YZ}}{\sqrt{(1 - r_{XZ}^2)(1 - r_{YZ}^2)}}.$$

La corrélation *conditionnelle* est celle calculée à l'intérieur de strates de Z (par exemple par poste).

```
def partial_corr(x, y, Z):
    Z = np.column_stack([np.ones(len(x)), Z])
    rx = x - Z @ np.linalg.lstsq(Z, x, rcond=None)[0] # résidus de x sur Z
    ry = y - Z @ np.linalg.lstsq(Z, y, rcond=None)[0]
    return stats.pearsonr(rx, ry)[0]
print(stats.pearsonr(D.temps, D.perf)[0]) # brute
print(partial_corr(D.temps.values, D.perf.values, D[["poste"]].values))
```

(La bibliothèque pingouin offre `pingouin.partial_corr` avec p-value et IC.) **Limites** : suppose des effets *linéaires* de Z ; ne corrige que les confondants inclus ; suppose que Z est mesuré sans erreur ; contrôler une mauvaise variable (médiateur, collisionneur) peut *créer* un biais.

23.2 Pourquoi l'association ne prouve pas la cause

- **Confusion** : un tiers influence X et Y (poste, âge, régime d'entraînement).
- **Causalité inverse** : Y cause X (jouer plus de matchs améliore une mesure physique).
- **Biais de sélection** : seuls certains joueurs sont observés (ceux invités au Combine, ceux qui ont joué assez d'actions).
- **Collisionneur** : conditionner sur un effet commun crée une fausse association.
- **Hasard**, surtout avec de nombreux tests.

Une **expérience** (assignation aléatoire du « traitement ») neutralise la confusion en moyenne ; une **étude observationnelle** ne le peut pas, quelle que soit la taille des données.

Application sportive / NFL

Une association entre performance physique mesurée avant la draft et performance en match ne prouve pas que « devenir plus rapide améliorerait le jeu » : la sélection pour la draft, le poste, le temps de jeu et l'entraînement jouent. Formule alors une conclusion en termes d'*association conditionnelle* (« à poste et gabarit comparables, les joueurs plus rapides tendent à... ») et liste les confondants non mesurés.

Sixième partie

Analyse exploratoire avancée (EDA)

Méthodologie complète d'une EDA

L'**analyse exploratoire des données** (EDA) est un processus ordonné qui vise à comprendre les données *avant* de modéliser, à repérer les problèmes, et à formuler des hypothèses. Voici une démarche en douze étapes, chacune avec le code Pandas typique.

#	Étape	Code ou action
1	Question et unité	écrire : « une ligne = ... »
2	Fichiers, schémas, dictionnaire	lire la documentation ; lister tables, clés, unités
3	Types et dimensions	<code>df.shape, df.dtypes, df.info()</code>
4	Valeurs manquantes	<code>df.isna().mean()</code>
5	Doublons	<code>df.duplicated(subset=cles).sum()</code>
6	Identifiants et clés de jointure	<code>df[cle].is_unique, nunique()</code>
7	Distributions	<code>describe()</code> , histogrammes, ECDF
8	Anomalies	valeurs impossibles, extrêmes, unités
9	Relations uni/bivariées	corrélations, boxplots, tableaux croisés
10	Interactions et sous-groupes	<code>groupby</code> , graphiques par poste, saison
11	Hypothèses	les écrire, séparer confirmatoire et exploratoire
12	Limites et conclusions	documenter, noter les décisions de nettoyage

```
def audit(df, cles=None):
    print("dimensions :", df.shape)
    print(df.dtypes)
    print("manquants (%) :\n", (df.isna().mean()*100).round(1).sort_values(ascending=False))
    if cles: print("doublons sur cles :", df.duplicated(subset=cles).sum())
    print(df.describe(include="all").T)
```

Attention

Une EDA n'est pas une chasse aux p-values. Chaque graphique ou test supplémentaire augmente le risque de « découverte » due au hasard ; note ce que tu as exploré, pas seulement ce que tu as trouvé. Garde aussi les décisions de nettoyage *dans le code*, jamais dans des modifications manuelles de fichiers.

Qualité des données

25.1 Données manquantes

Mécanismes de données manquantes

- **MCAR** (*missing completely at random*) : la probabilité d'absence est indépendante de toutes les variables. Le cas le plus bénin.
- **MAR** (*at random*) : l'absence dépend de variables *observées* (par exemple le test n'est pas passé pour certains postes).
- **MNAR** (*not at random*) : l'absence dépend de la valeur manquante elle-même (un joueur évite le test qu'il réussirait mal).

On ne peut *pas* prouver le mécanisme à partir des données seules ; on raisonne sur le processus de collecte et on examine si l'absence est associée à d'autres variables.

```
df = pd.DataFrame({"poste": rng.choice(["WR", "CB", "OL"], 300), "temps40": rng.normal(4.8, .3, 300)})
df.loc[(df.poste == "OL") & (rng.random(300) < .6), "temps40"] = np.nan # MAR illustratif
print(df.groupby("poste")["temps40"].apply(lambda s: s.isna().mean())) # absence selon poste
```

Stratégie	Quand	Risque
Suppression des lignes	peu de manquants, MCAR plausible	perte de puissance, biais si MAR/MNAR
Imputation médiane/moyenne	prétraitement simple de modèles	réduit artificiellement la variance
Imputation par groupe ou KNN	MAR plausible	dépend du modèle d'imputation
Indicateur d'absence	l'absence est informative	peut fuir de l'information
Imputation multiple	inférence statistique sérieuse	plus complexe

Attention

Imputer avant la séparation entraînement/test est une fuite de données : apprend les paramètres d'imputation sur l'entraînement uniquement (`SimpleImputer` dans un `Pipeline`).

25.2 Autres problèmes de qualité

- **Doublons** : lignes identiques ou clés répétées par erreur (`uplicated`, `drop_duplicates`) ; vérifier si un doublon est une erreur ou une répétition légitime.
- **Erreurs de type** : nombres lus comme texte, dates non converties (`pd.to_numeric(errors="coerce")`, `pd.to_datetime`).
- **Unités incohérentes** : pieds/mètres, yards/mètres, secondes/millisecondes ; vérifier les ordres de grandeur.

- **Valeurs impossibles** : taille négative, vitesse surhumaine, coordonnées hors du terrain.
- **Biais de mesure** : capteurs différents selon stade ou saison, étalonnage dérivant.
- **Fuite depuis la cible** (*target leakage*) : une variable contient directement ou indirectement le résultat à prédire (calculée après l'événement). Dès qu'une variable est « trop belle », cherche la fuite.

```
d = pd.DataFrame({"h": ["72", "75", "n/a", "71"], "date": ["2025-09-07", "2025-09-14", "x", "2025-09-21"]})
d["h"] = pd.to_numeric(d["h"], errors="coerce") # n/a -> NaN
d["date"] = pd.to_datetime(d["date"], errors="coerce") # x -> NaT
print(d.isna().sum())
assert (d["h"].dropna().between(55, 90)).all() # controle de plausibilite
```

Jointures et données multi-tables

Clés

Une **clé primaire** identifie de façon unique une ligne d'une table (`player_id` dans la table `joueurs`). Une **clé étrangère** référence la clé primaire d'une autre table (`player_id` dans la table des actions). La **cardinalité** décrit la correspondance : un-à-un, un-à-plusieurs, plusieurs-à-plusieurs.

Jointure	Effet
<code>inner</code>	garde seulement les clés présentes dans les deux tables
<code>left</code>	garde toutes les lignes de gauche, <code>NaN</code> si absent à droite
<code>right, outer</code>	symétrique ; union des clés
<code>concat</code>	empile des tables (axe 0) ou les colle côte à côte (axe 1)

```
joueurs = pd.DataFrame({"player_id": [1, 2, 3], "poste": ["WR", "CB", "OL"], "temps40": [4.4, 4.5, 5.2]})
actions = pd.DataFrame({"player_id": [1, 1, 2, 2, 2, 9], "gain": [5, 12, 0, 3, 7, 4]})
m = actions.merge(joueurs, on="player_id", how="left",
                  validate="many_to_one", # leve une erreur si joueurs a des doublons de cle
                  indicator=True) # colonne _merge : both / left_only
print(m); print(m["_merge"].value_counts())
```

Contrôles obligatoires après une jointure :

1. le nombre de lignes est celui attendu (`len(m)` contre `len(actions)`);
2. aucune duplication involontaire (`validate=...`);
3. la part de lignes non appariées (`indicator=True`) est comprise et justifiée ;
4. les types des clés sont identiques (entier contre texte = zéro correspondance silencieuse) ;
5. les valeurs de la table jointe ne sont pas comptées plusieurs fois dans les agrégats suivants.

Attention

Une jointure plusieurs-à-plusieurs *multiplie* les lignes (a lignes contre b lignes pour la même clé donnent $a \times b$ lignes). Moyenner ensuite revient à pondérer artificiellement. Si l'on joint des mesures physiques (une ligne par joueur) à du tracking (plusieurs lignes par joueur), chaque joueur est répété ; la *taille effective* de l'échantillon reste le nombre de joueurs.

Application sportive / NFL

Exemple de schéma (illustratif) : table `joueurs` (une ligne par joueur), `matchs` (une par match), `evenements` (une par action, clés `match_id`, `player_id`), `mesures` (une par joueur, éventuellement plusieurs sessions). Dessine ce schéma avant de coder une jointure.

Agrégations et analyse par groupe

```
ev = pd.DataFrame({"player_id": rng.integers(1, 6, 200),
                  "poste": None, "saison": rng.choice([2023, 2024], 200),
                  "gain": rng.gamma(2, 3, 200)})
ev["poste"] = ev.player_id.map({1: "WR", 2: "WR", 3: "CB", 4: "CB", 5: "OL"})

agg = ev.groupby(["poste", "saison"]).agg(n=("gain", "size"),
                                         moyenne=("gain", "mean"),
                                         mediane=("gain", "median")) # agregations nommees
ev["gain_centre"] = ev["gain"] - ev.groupby("player_id")["gain"].transform("mean") # meme
longueur
pv = ev.pivot_table(index="poste", columns="saison", values="gain", aggfunc="median")
```

- `groupby` découpe en groupes ; `agg` réduit chaque groupe à une ligne ; `transform` renvoie une valeur par ligne d'origine (centrage par groupe, rangs, proportions) ; `pivot_table` produit un tableau croisé agrégé.
- `observed=True` avec des variables catégorielles Pandas évite des groupes vides.
- Afficher toujours l'effectif (`size`) et regarder les distributions, pas seulement les moyennes.

27.1 Choisir l'unité d'analyse

Si la question porte sur les *joueurs*, l'unité est le joueur : on agrège les actions par joueur (médiane, moyenne *et* nombre d'actions) puis on analyse une ligne par joueur. Si la question porte sur la variation entre actions d'un même joueur, on garde le niveau action, avec un modèle qui tient compte des regroupements.

Attention

Une agrégation excessive peut masquer des phénomènes (paradoxe de Simpson, Chapitre 32), une agrégation insuffisante gonfle artificiellement n . Les moyennes de groupes de tailles très différentes n'ont pas la même précision : une moyenne sur 3 actions est bien moins fiable qu'une sur 300.

Séries temporelles et données de tracking

28.1 Notions de base

Une **série temporelle** est une suite de mesures ordonnées dans le temps ; le **pas de temps** est l'intervalle entre deux mesures (par exemple 0,1 s pour un tracking à 10 Hz). Avant tout calcul : trier par temps *au sein de chaque joueur et chaque action*, vérifier la régularité des pas, et vérifier l'absence de trous.

```
tr = pd.DataFrame({"t": np.arange(0, 5, .1)})
tr["x"] = 5*tr.t + rng.normal(0, .15, len(tr)) # position en yards
tr["y"] = 2*np.sin(tr.t) + rng.normal(0, .15, len(tr))
tr = tr.sort_values("t") # ordre temporel obligatoire
dt = tr.t.diff() # pas de temps
print(dt.describe()) # regulier ?
tr["vx"] = tr.x.diff() / dt # vitesse : diff finie
tr["vy"] = tr.y.diff() / dt
tr["speed"] = np.hypot(tr.vx, tr.vy) # norme de la vitesse
tr["acc"] = tr.speed.diff() / dt # acceleration
tr["speed_lisse"] = tr.speed.rolling(5, center=True, min_periods=1).mean()
```

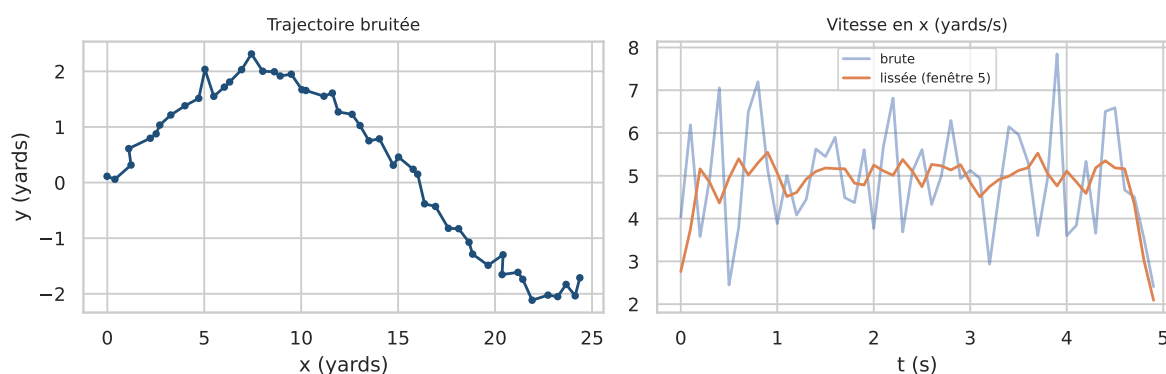


FIG. 28.1 : Données synthétiques : trajectoire bruitée et vitesse en x avant/après lissage par moyenne glissante sur 5 points. Dériver amplifie le bruit.

28.2 Notions détaillées

- **Différences et variations** : `diff()` calcule $x_t - x_{t-1}$; diviser par le pas donne une dérivée numérique. La première valeur est `NaN`.
- **Fenêtres glissantes** : `rolling(window).mean()`, `.median()`, `.std()` ; `center=True` évite de décaler le signal ; une fenêtre trop large lisse des événements réels, trop petite laisse le bruit.
- **Lissage** : moyenne mobile, `ewm`, filtre de Savitzky–Golay (`scipy.signal.savgol_filter`) qui préserve mieux les pics.

- **Synchronisation** : aligner plusieurs joueurs sur le même instant ou sur un *événement de référence* (début de l'action) plutôt que sur l'horloge brute ; vérifier que les pas sont les mêmes.
- **Détection d'événements** : seuils sur une variable dérivée (par exemple accélération > seuil), via des comparaisons avec `shift()` pour repérer un franchissement.
- **Agrégation temporelle** : maximum de vitesse, temps passé au-dessus d'un seuil, par action ; choisir l'unité pertinente.
- **Trajectoires** : tracer (x, y) avec un repère à échelle égale.

28.3 Autocorrélation et dépendance

Autocorrélation

L'autocorrélation de décalage k est la corrélation d'une série avec elle-même décalée de k pas : $\rho_k = \text{corr}(x_t, x_{t-k})$.

```
print(tr.speed.dropna().autocorr(lag=1)) # pandas
v = tr.speed.dropna().values - tr.speed.dropna().mean()
acf1 = np.sum(v[1:]*v[:-1]) / np.sum(v*v) # estimation simple du lag 1
print(acf1)
```

Pourquoi les observations ne sont pas indépendantes

Si ton joueur va vite à l'image t , il va probablement vite à $t+1$. Dix mille images d'un même joueur ne valent pas dix mille informations indépendantes. Pour une série à autocorrélation ρ (modèle autorégressif d'ordre 1), la taille *effective* est approximativement $n_{\text{eff}} \approx n \frac{1-\rho}{1+\rho}$. Avec $n = 1000$ et $\rho = 0,9$, $n_{\text{eff}} \approx 53$.

Conséquence : tests et intervalles qui supposent l'indépendance sont *trop optimistes* sur de telles données. Solutions : agréger à un niveau pertinent (une valeur par action, par joueur), utiliser des erreurs-types groupées, des modèles à effets mixtes, ou un bootstrap par blocs/clusters.

Attention

Ne calcule des indicateurs physiques (vitesse, accélération, distance) que si les variables nécessaires, leurs *unités* et leur définition sont documentées dans le dictionnaire de données. Une vitesse fournie par le capteur et une vitesse recalculée à partir des positions peuvent différer (lissage interne, unité, orientation).

Septième partie

Statistiques spécifiques au sport et au NFL Big Data Bowl

Construire de bonnes questions de recherche

Une bonne question est le résultat d'un tri. Chaque critère ci-dessous est un filtre ; une idée qui échoue à l'un d'eux doit être reformulée ou abandonnée.

Critère	Question de contrôle
Intéressante	Quelqu'un (entraîneur, analyste) changerait-il d'avis ou d'action selon la réponse ?
Testable	Peut-on la formuler par une comparaison, une relation ou une prédiction mesurable ?
Pertinente	Correspond-elle à une réalité du football américain (rôles, contraintes de jeu) ?
Compatible avec les données	Les variables nécessaires existent-elles, avec la bonne unité et la bonne granularité ?
Originale	Une recherche rapide de travaux et notebooks existants montre-t-elle un angle nouveau ?
Démontrable	Peut-on la traiter avec les données disponibles en respectant leurs limites ?

Exemples de formulations possibles (aucun résultat affirmé)

- Les qualités physiques mesurées avant la draft sont-elles associées, à poste comparable, à des différences de mouvement observées en match ?
- Cette association varie-t-elle selon le poste ou le contexte de jeu ?
- La capacité de changement de direction, mesurée par tracking, ajoute-t-elle de l'information prédictive par rapport à la seule vitesse en ligne droite ?
- Une mesure physique unique est-elle plus informative qu'une combinaison de plusieurs mesures ?

Passage d'une idée à un plan. (1) *Unité d'analyse*; (2) *variables* (explicative, cible, confondants, identifiants); (3) *population et filtres* (postes, minimum d'actions); (4) *méthode primaire* choisie avant l'exploration; (5) *contrôles* (sous-groupes, sensibilité); (6) *ce qui infirmerait l'hypothèse*.

Attention

Une question séduisante mais impossible à répondre avec les données disponibles (par exemple causale à partir de données purement observationnelles) doit être reformulée en question d'association ou de prédiction.

Analyse de performance et comparaisons équitables

- **Comparer des joueurs ou des postes** : comparer seulement des unités comparables (poste, rôle, contexte). Un défenseur de ligne et un receveur n'ont ni la même morphologie ni les mêmes tâches.
- **Âge, expérience** : si disponibles, ce sont des confondants plausibles ; les ajuster ou stratifier.
- **Contexte de jeu** : adversaire, situation, formation, météo, ... influencent la performance indépendamment du joueur.
- **Effectifs par groupe** : 30 actions ne permettent pas de comparer des profils fins ; afficher n .
- **Variabilité individuelle** : l'écart entre joueurs et l'écart entre actions d'un même joueur sont deux sources différentes ; la seconde peut dominer.
- **Incertitude des classements** : quand les valeurs reposent sur peu d'observations, un classement est instable.

Estimer l'incertitude d'un classement par bootstrap.

```
perf = pd.DataFrame({"joueur": rng.integers(0, 8, 160), "score": rng.normal(0, 1, 160)})
def classement(df):
    return df.groupby("joueur")["score"].mean().rank(ascending=False)
rangs = pd.DataFrame([classement(perf.sample(frac=1, replace=True, random_state=i))
                      for i in range(1000)])
print(rangs.quantile([0.05, 0.95]).T) # intervalle de rang plausible par joueur
```

Ici toutes les données sont du bruit pur : pourtant un classement existe, et les intervalles de rang sont très larges, ce qui illustre que *tout classement produit des gagnants, même sans écart réel*. (Pour un vrai bootstrap par joueur, on rééchantillonne *avec les joueurs comme unités*.)

Attention

Classer des joueurs sur *une seule métrique* produit des conclusions simplistes : la métrique peut dépendre du rôle, du contexte, et de l'effectif. Utilise plusieurs métriques, des intervalles d'incertitude, et le contexte. Pour les petits effectifs, des méthodes de **rétrécissement** (modèles hiérarchiques, estimations « shrinkées » vers la moyenne du groupe) donnent des estimations plus stables que les moyennes brutes.

Mesures de tracking et géométrie

31.1 Coordonnées et unités

Les positions sont des coordonnées (x, y) dans un repère lié au terrain. Avant tout calcul, vérifier : l'**unité** (yards, mètres), l'**origine** et l'orientation des axes, et si les coordonnées sont **normalisées** selon le sens de l'attaque. Un angle peut être défini depuis des axes différents selon les jeux de données : lire le dictionnaire.

```
p = np.array([[0.0, 0.0], [3.0, 4.0], [6.0, 8.0]]) # trois positions successives (yards)
dep = np.diff(p, axis=0) # déplacements successifs
dist_pas = np.linalg.norm(dep, axis=1) # distance euclidienne pas a pas
print(dist_pas, dist_pas.sum()) # [5. 5.] 10.0 (distance parcourue)
print(np.linalg.norm(p[-1] - p[0])) # déplacement net 10.0
```

Distance parcourue (somme des pas) et **déplacement** (distance entre début et fin) diffèrent dès que la trajectoire n'est pas rectiligne.

31.2 Vitesse, accélération, direction

$$\text{vitesse}_t = \frac{\|p_t - p_{t-1}\|}{\Delta t}, \quad \text{accélération}_t = \frac{\text{vitesse}_t - \text{vitesse}_{t-1}}{\Delta t},$$

$$\theta_t = \text{atan2}(\Delta y_t, \Delta x_t), \quad \Delta \theta_t = ((\theta_t - \theta_{t-1} + \pi) \bmod 2\pi) - \pi.$$

La dernière formule ramène la variation d'angle dans $[-\pi, \pi]$: sans cette étape, passer de 179° à -179° donnerait une rotation fausse de 358° au lieu de 2° .

```
dx, dy, dt = np.diff(tr.x), np.diff(tr.y), np.diff(tr.t)
theta = np.arctan2(dy, dx)
dtheta = (np.diff(theta) + np.pi) % (2*np.pi) - np.pi # variation d'angle, bien enroulee
print(np.degrees(np.abs(dtheta)).max())
```

31.3 Distances entre joueurs, voisinage

```
from scipy.spatial.distance import cdist
A = rng.uniform(0, 50, (11, 2)); B = rng.uniform(0, 50, (11, 2)) # deux equipes a un instant
Dm = cdist(A, B) # matrice 11 x 11 de distances euclidiennes
print(Dm.min(axis=1)) # distance a l'adversaire le plus proche, par joueur de A
print(Dm.argmin(axis=1)) # indice de cet adversaire
```

Dans un tracking réel, calculer ces distances *image par image* et regrouper par action ; le voisin le plus proche change au cours du temps.

31.4 Bruit, pas de temps, limites

- **Dérivation numérique amplifie le bruit** : une erreur de position de $\pm 0,1$ yard avec un pas de 0,1 s donne une erreur de vitesse de l'ordre de ± 1 yard/s, et bien pire pour l'accélération (voir la figure du chapitre précédent).
- **Pas de temps** : un pas trop grand manque les changements brusques ; un pas trop petit amplifie le bruit. Comparer plusieurs pas ou lissages avant de conclure.
- **Données irrégulières ou incomplètes** : si des images manquent, la différence Δt n'est pas constante ; toujours diviser par le *vrai* Δt ; ne pas interpoler sans y réfléchir.
- **Cohérence des sources** : vérifier que la vitesse recalculée est cohérente avec celle éventuellement fournie.

Les analyses statistiques (Seaborn, SciPy) s'appliquent ensuite à des *indicateurs par action ou par joueur*, pas aux images brutes.

Éviter les conclusions trompeuses

Piège		Mécanisme	Parade
Paradoxe de Simpson		tendance inversée par agrégation de groupes hétérogènes	analyser par groupe, ajuster, tracer
Biais de sélection		l'échantillon observé n'est pas représentatif	se demander <i>qui est absent et pourquoi</i>
Biais du survivant		on n'observe que ce qui a « survécu » (joueurs encore en activité)	inclure les sorties, ou limiter la conclusion
Corrélation ≠ causalité		confusion, causalité inverse	ajustement, raisonnement causal, prudence
Multiplicité		beaucoup de tests, quelques faux positifs	corrections, pré-spécification
Fuite de données		information de test dans l'apprentissage ou cible indirecte	pipelines, séparation par groupes
Sous-groupes		découvertes sur de petits groupes	effectifs, intervalles, réplication
Trop peu d'observations		estimations instables	intervalles, prudence

Simpson en chiffres (valeurs illustratives, vérifiées par le code ci-dessous). L'équipe A réussit $18/20 = 0,90$ des tentatives faciles et $54/180 = 0,30$ des difficiles, soit $72/200 = 0,36$ au global. L'équipe B réussit $150/180 \approx 0,83$ des faciles et $4/20 = 0,20$ des difficiles, soit $154/200 = 0,77$ au global. A fait mieux dans *chaque* situation, mais B paraît bien meilleure au global parce que ses tentatives sont surtout faciles. Le contexte est le confondant. La Figure du Chapitre 7 montre la même chose graphiquement.

```
tab = pd.DataFrame({"equipe": ["A", "A", "B", "B"], "situation": ["facile", "difficile"]*2,
                    "tentatives": [20, 180, 180, 20], "reussites": [18, 54, 150, 4]})
tab["taux"] = tab.reussites / tab.tentatives
print(tab) # A meilleure dans chaque situation
print(tab.groupby("equipe")[["reussites", "tentatives"]].sum().assign(
    taux=lambda t: t.reussites / t.tentatives)) # taux global
```

Attention

Surinterprétation des sous-groupes : si tu coupes les données en 20 sous-groupes, tu trouveras « un effet » dans au moins un par hasard. Annonce le nombre de sous-groupes explorés, corrige, et traite les résultats comme des *hypothèses* à reproduire.

Application sportive / NFL

Un ensemble de joueurs ayant des données complètes peut déjà être sélectionné (joueurs ayant été assez utilisés). Une conclusion sur « les joueurs » en général est alors un cas de biais de sélection ; formule-la pour la population *effectivement observée*.

Huitième partie

Machine Learning pour compléter l'analyse statistique

Quand utiliser le Machine Learning ?

Objectif	Question type	Outils
Description	Que contiennent les données ?	statistiques descriptives, graphiques
Inférence	Que peut-on dire de la population ? Un effet est-il compatible avec le hasard ?	tests, IC, régression pour l'inférence
Exploration	Quelles structures, quelles hypothèses ?	EDA, ACP, clustering
Prédiction	Peut-on prédire Y sur de nouvelles données ?	modèles prédictifs, validation hors échantillon
Causalité	Que se passerait-il si l'on intervenait ?	expériences, méthodes causales (hypothèses fortes)

Règle de choix. Si la question est « combien, avec quelle incertitude, ajusté pour quoi ? », privilégier une méthode statistique interprétable. Si la question est « quelle précision de prédiction sur des cas nouveaux ? », un modèle prédictif validé est approprié. Un modèle prédictif excellent peut ne rien dire sur les causes ; une méthode statistique interprétable peut très mal prédire. Les deux approches sont complémentaires : le ML peut servir à repérer des relations non linéaires ou des interactions, que l'on examine ensuite avec des outils statistiques.

Attention

Une précision élevée n'est pas une explication. Si ta conclusion contient « parce que », demande-toi ce qui, dans la démarche, la justifie.

Validation correcte des modèles

34.1 Séparation et validation croisée

- **Entraînement** : apprendre les paramètres. **Validation** : choisir hyperparamètres et modèle. **Test** : évaluer une seule fois à la fin, comme estimation de la performance réelle.
- **Validation croisée** (k plis) : l'ensemble est coupé en k parties; on entraîne sur $k - 1$ et on évalue sur la dernière, k fois; on moyenne et on regarde la dispersion.

34.2 Fuite de données et pipelines

La **fuite de données** survient quand de l'information venant des données d'évaluation influence l'entraînement : standardiser avant la séparation, imputer avec la moyenne globale, sélectionner des variables sur toutes les données, ou utiliser une variable qui dépend de la cible. La parade est de mettre *tout le prétraitement dans un Pipeline*, qui n'apprend ses paramètres que sur les données d'entraînement de chaque pli.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score, GroupKFold

n = 300
data = pd.DataFrame({"temps": rng.normal(4.7, .25, n), "poids": rng.normal(100, 15, n),
                    "poste": rng.choice(["WR", "CB", "OL"], n), "joueur": rng.integers(0, 60, n)})
data.loc[rng.random(n) < .1, "temps"] = np.nan
y = 50 - 8*(data.temps.fillna(4.7) - 4.7) + rng.normal(0, 3, n)

prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())]), ["temps", "poids"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["poste"])])
pipe = Pipeline([("prep", prep), ("mod", Ridge(alpha=1.0))])

gkf = GroupKFold(n_splits=5)
scores = cross_val_score(pipe, data[["temps", "poids", "poste"]], y,
                        groups=data.joueur, cv=gkf, scoring="neg_root_mean_squared_error")
print(-scores.mean(), scores.std())
```

Explication : `ColumnTransformer` applique des traitements différents par colonne; `Pipeline` les enchaîne avec le modèle; `groups=` indique les unités qui ne doivent pas être coupées entre entraînement et validation; `scoring` utilise la convention « plus grand = mieux » de Scikit-learn (d'où `neg_`). (Ici la cible est construite avec des valeurs imputées pour l'illustration, ne pas imiter en pratique.)

34.3 Séparation adaptée à la structure

Une séparation aléatoire ordinaire place des lignes d'un même joueur dans l'entraînement et le test : le modèle « reconnaît » le joueur et l'on surestime la performance. Il faut séparer selon l'unité à laquelle on veut généraliser :

- `GroupKFold`, `GroupShuffleSplit` : généraliser à de *nouveaux joueurs* (ou nouveaux matchs, équipes).
- `TimeSeriesSplit` ou séparation temporelle : prédire le *futur* (entraîner sur les saisons passées, tester sur la plus récente).
- `StratifiedKFold` (ou `StratifiedGroupKFold`) : conserve la proportion des classes.

Attention

Pose-toi toujours la question : « sur quelle population nouvelle le modèle sera-t-il utilisé ? » La séparation doit imiter ce passage. Un modèle bon sur de nouvelles *actions de joueurs connus* peut être mauvais sur de *nouveaux joueurs*.

34.4 Métriques

Régression : MAE, RMSE, R^2 hors échantillon. Classification : exactitude (trompeuse si classes déséquilibrées), précision, rappel, F1, AUC, log-loss, calibration. Choisir la métrique selon le coût des erreurs, et toujours la comparer à une référence simple (prédire la moyenne ou la classe majoritaire).

Modèles et interprétation

Modèle	Idée	Points d'attention
Régression linéaire / Ridge / Lasso	somme pondérée ; régularisation pénalise les coefficients (ℓ_2 ridge, ℓ_1 lasso, qui annule certains)	échelle des variables à standardiser ; linéarité
Régression logistique	probabilité d'une classe par fonction logistique ; coefficients en log-odds	séparation parfaite, régularisation par défaut dans Scikit-learn
Arbre de décision / Random Forest	suites de questions oui/non moyenne de nombreux arbres sur échantillons bootstrap	très instable, sur-apprentissage robuste, moins interprétable, importance biaisée
Gradient boosting	arbres construits séquentiellement pour corriger les erreurs	performant, nombreux hyperparamètres, risque de sur-apprentissage
KNN	prédit à partir des k plus proches voisins	exige une mise à l'échelle ; lent sur gros jeux

Fiche méthode

Objectif	Prédire Y à partir de variables explicatives, et estimer leur importance relative.
Quand l'utiliser	Relations non linéaires, interactions, nombreuses variables, objectif de prédiction.
Quand NE PAS l'utiliser	Quand on a besoin d'un effet interprétable ajusté (préférer une régression), de très peu de données, ou d'une conclusion causale.
Hypothèses	Observations suffisamment indépendantes (ou séparation par groupes), variables disponibles au moment de la prédiction.
Limites	Sur-apprentissage ; importance corrélée aux variables redondantes ; interprétation causale non justifiée.
Interprétation	Comparer à une référence, regarder la dispersion entre plis, vérifier la généralisation.

```

from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.inspection import permutation_importance
from sklearn.model_selection import GroupShuffleSplit
Xd = pd.get_dummies(data[["temps", "poids", "poste"]].fillna(data[["temps"]].median()),
                    drop_first=True)
gss = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=0)
tr_i, te_i = next(gss.split(Xd, y, groups=data.joueur))
rf = RandomForestRegressor(n_estimators=200, min_samples_leaf=5, random_state=0).fit(Xd.iloc[tr_i],
                                          y.iloc[tr_i])
print(rf.score(Xd.iloc[te_i], y.iloc[te_i])) # R2 sur de nouveaux joueurs
pi = permutation_importance(rf, Xd.iloc[te_i], y.iloc[te_i], n_repeats=20, random_state=0)
print(pd.Series(pi.importances_mean, index=Xd.columns).sort_values(ascending=False))

```

Importance par permutation. On mélange aléatoirement *une* colonne sur les données d'évaluation et l'on mesure la dégradation de la performance : plus elle est grande, plus le

modèle *s'appuie* sur cette variable. Elle se calcule sur des données non vues, ce qui évite l'écueil de l'importance « par impureté » (biaisée en faveur des variables à nombreuses valeurs).

Attention

L'importance d'une variable **n'est pas une preuve de causalité**. Quand deux variables sont corrélées, le modèle peut s'appuyer sur l'une des deux : mélanger l'une est compensé par l'autre et son importance paraît faible ; l'importance se répartit de façon arbitraire. Regroupe les variables corrélées ou analyse-les ensemble.

Application sportive / NFL

Un modèle qui « utilise beaucoup » une mesure physique pour prédire une performance ne dit pas que cette qualité cause la performance : il dit qu'elle est informative *dans ces données*, éventuellement parce qu'elle remplace le poste ou le rôle.

Réduction de dimension et structures cachées

36.1 ACP (PCA)

L'**analyse en composantes principales** remplace des variables corrélées par de nouveaux axes non corrélés (**composantes**), ordonnés par la variance qu'ils capturent. La **variance expliquée** de la composante k est $\lambda_k / \sum_j \lambda_j$, où λ_k sont les valeurs propres de la matrice de covariance. Les **scores** sont les coordonnées des observations sur les composantes; les **chargements** (`components_`) indiquent la contribution des variables.

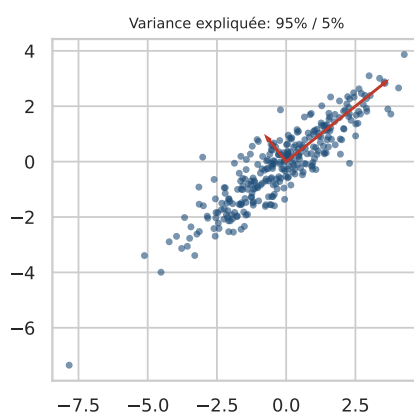


FIG. 36.1 : Nuage synthétique et deux composantes principales (flèches proportionnelles à l'écart-type porté).

Fiche méthode

Objectif

Résumer beaucoup de variables corrélées en peu d'axes, visualiser, réduire le bruit.

Quand l'utiliser

Variables nombreuses et corrélées, exploration, prétraitement.

Quand NE PAS l'utiliser

Variables sur des échelles différentes non standardisées; relation fortement non linéaire; quand on a besoin d'interpréter chaque variable d'origine.

Hypothèses

Relations à peu près linéaires; variance $\approx$ information (discutable).

Limites

Composantes difficiles à interpréter; sensible aux extrêmes et à l'échelle; la variance n'est pas toujours la partie utile du signal.

Interprétation

Examiner la variance expliquée cumulée et les chargements; ne pas sur-interpréter les axes.

```
from sklearn.decomposition import PCA
Xp = rng.multivariate_normal([0, 0, 0], [[3, 2, 0], [2, 2, 0], [0, 0, 1]], 300)
Xs = StandardScaler().fit_transform(Xp) # standardiser d'abord !
pca = PCA(n_components=None).fit(Xs)
print(pca.explained_variance_ratio_.round(3), pca.explained_variance_ratio_.cumsum().round(3))
scores = pca.transform(Xs); print(pca.components_.round(2))
```

36.2 Clustering

Le **clustering** (par exemple *k*-means) cherche des groupes d'observations proches ; **c'est un outil exploratoire**, sans garantie que les groupes aient un sens. Il exige de choisir le nombre de groupes, la mesure de distance et la mise à l'échelle.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
for k in (2, 3, 4):
    lab = KMeans(n_clusters=k, n_init=10, random_state=0).fit_predict(Xs)
    print(k, silhouette_score(Xs, lab)) # cohesion/separation, entre -1 et 1
```

Attention

Un algorithme trouvera toujours des groupes, même dans du bruit uniforme. Un groupe «découvert» diffère d'un groupe *observé* (poste, équipe). Valide la stabilité (rééchantillonnage), la signification sportive, et ne pas comparer ensuite les groupes avec un test classique comme si leur définition était indépendante des données (circularité). Ne recommande pas ACP ou clustering automatiquement : demande d'abord quelle *question* ils servent.

Neuvième partie

Python et bibliothèques à maîtriser

Ce chapitre est une référence : pour chaque bibliothèque, les fonctions réellement utiles à l'analyse, avec objectif, paramètres importants, données attendues, forme du résultat et erreur fréquente. Les exemples d'usage détaillés sont dans les chapitres correspondants. Les noms de paramètres peuvent évoluer entre versions : *vérifie la documentation de ta version* (`help(fonction)`).

NumPy et Pandas

37.1 NumPy

Fonction	Objectif	Paramètres clés	Erreur fréquente
<code>np.array</code> , <code>reshape</code>	créer, changer la forme	<code>(-1, 1)</code> pour une colonne	oublier la 2D pour Scikit-learn
indexation, masques booléens	sélectionner (<code>a[a > 3]</code>)	conditions combinées avec <code>&</code> , <code> </code>	utiliser <code>and/or</code>
<code>np.mean</code> , <code>median</code> , <code>std</code> , <code>var</code> , <code>percentile</code>	statistiques	<code>axis</code> , <code>ddof</code>	<code>ddof=0</code> par défaut pour <code>std/var</code>
<code>np.nanmean</code> , <code>nanstd</code> , <code>nanpercentile</code>	ignorer les NaN	mêmes	<code>np.mean</code> renvoie nan
<code>np.corrcoef</code>	matrice de corrélation	<code>rowvar</code>	variables en lignes vs colonnes
<code>np.diff</code> , <code>np.gradient</code>	dérivées numériques	<code>axis</code> , d'espacement	ignorer un pas irrégulier
<code>np.linalg.norm</code>	distances, normes	<code>axis</code>	oublier l'axis
<code>np.arctan2</code> , <code>np.hypot</code>	angles, norme	ordre (y, x) pour <code>arctan2</code>	inverser x et y
<code>np.random.default_rng</code>	tirages reproductibles	graine	utiliser l'ancien <code>np.random.seed</code> partout

37.2 Pandas

Fonction	Objectif	Paramètres clés	Erreur fréquente
<code>read_csv</code> , <code>info</code> , <code>dtypes</code> , <code>describe</code>	lire, inspecter résumé statistique	<code>dtype</code> , <code>parse_dates</code> , <code>include="all"</code> , <code>percentiles</code>	types non vérifiés ignorer l'effectif
<code>isna</code> , <code>dropna</code> , <code>fillna</code>	manquants	<code>subset</code> , <code>how</code>	imputer avant la séparation
<code>uplicated</code> , <code>drop_duplicates</code>	doublons	<code>subset</code> , <code>keep</code>	doublons sur toutes colonnes seulement
<code>groupby</code> , <code>agg</code> , <code>transform</code>	par groupe	agrégations nommées	confondre <code>agg</code> et <code>transform</code>
<code>merge</code> , <code>join</code> , <code>concat</code>	combiner	<code>on</code> , <code>how</code> , <code>validate</code> , <code>indicator</code>	duplication silencieuse
<code>pivot_table</code> , <code>crosstab</code>	tableaux croisés	<code>aggfunc</code> , <code>normalize</code>	mauvaise normalisation
<code>to_datetime</code> , <code>diff</code> , <code>rolling</code> , <code>shift</code>	dates, temporel	<code>errors</code> , fenêtre, <code>center</code>	oublier de trier
<code>corr</code> , <code>cov</code>	corrélations	<code>method</code> , <code>min_periods</code>	n variable selon les paires
<code>value_counts</code> , <code>cut</code> , <code>qcut</code>	fréquences, classes	<code>normalize</code> , <code>bins</code>	classes arbitraires

Matplotlib et Seaborn

38.1 Matplotlib

Une **figure** (Figure) contient un ou plusieurs **axes** (Axes). L'interface orientée objet est la plus robuste :

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True) # 1 ligne, 2 colonnes
axes[0].plot(tr.x, tr.y, "o-", ms=3, label="trajectoire")
axes[0].set_aspect("equal"); axes[0].set_xlabel("x (yards)"); axes[0].set_ylabel("y (yards)")
axes[0].annotate("depart", xy=(tr.x.iloc[0], tr.y.iloc[0]), xytext=(2, 2),
                arrowprops=dict(arrowstyle="->"))
axes[0].legend(); axes[1].hist(tr.speed.dropna(), bins=20)
fig.suptitle("Titre"); fig.tight_layout(); fig.savefig("figure.png", dpi=200)
```

Éléments utiles : `subplots` (grille), `set_xlabel/set_title`, `legend`, `annotate`, `axvline/axhline`, `fill_between` (bandes d'incertitude), `savefig(dpi=...)`.

38.2 Seaborn

Fonction	Usage	Remarque
<code>histplot</code> , <code>kdeplot</code>	distribution	<code>bins</code> , <code>stat</code> , <code>hue</code> , <code>common_norm</code>
<code>ecdfplot</code>	répartition empirique	comparer groupes sans lissage
<code>boxplot</code> , <code>violinplot</code>	distribution par groupe	ajouter les points si <i>n</i> petit
<code>scatterplot</code> , <code>regplot</code>	relation entre deux variables	<code>alpha</code> , <code>lowess=True</code> , <code>ci</code>
<code>lineplot</code>	tendance temporelle, IC	agrège par défaut les répétitions
<code>heatmap</code>	matrices	<code>vmin</code> , <code>vmax</code> , <code>center</code> , <code>annot</code>
<code>pairplot</code>	nuages deux à deux	<code>hue</code> , <code>corner</code>
<code>pointplot</code>	estimations avec IC	montre l'incertitude

`hue` colore par groupe ; `col/row` (`displot`, `relplot`, `catplot`) créent des facettes par groupe.

SciPy, statsmodels et Scikit-learn

39.1 SciPy (`scipy.stats`)

Fonction	Rôle	Résultat
<code>norm</code> , <code>t</code> , <code>expon</code> , <code>poisson</code> , <code>binom</code> (<code>pdf</code> , <code>cdf</code> , <code>ppf</code> , <code>rvs</code>)	distributions	valeurs, probabilités, quantiles
<code>pearsonr</code> , <code>spearmanr</code> , <code>kendalltau</code> , <code>pointbiserialr</code>	corrélations	<code>statistic</code> , <code>pvalue</code> ; IC pour Pearson
<code>shapiro</code> , <code>normaltest</code> , <code>kstest</code> , <code>probplot</code>	normalité	statistique, p-value; coordonnées Q-Q
<code>ttest_ind</code> , <code>ttest_rel</code> , <code>mannwhitneyu</code> , <code>wilcoxon</code>	deux groupes	statistique, p-value
<code>f_oneway</code> , <code>kruskal</code> , <code>levene</code> , <code>tukey_hsd</code>	plusieurs groupes	statistique, p-value, comparaisons
<code>chi2_contingency</code> , <code>fisher_exact</code> , <code>contingency.</code> <code>association</code>	catégorielles	statistique, p, ddl, attendus; odds ratio; V
<code>bootstrap</code> , <code>permutation_test</code>	rééchantillonnage	IC; p-value et distribution nulle
<code>zscore</code> , <code>iqr</code> , <code>median_</code> <code>abs_deviation</code> , <code>skew</code> , <code>kurtosis</code>	descriptif robuste	valeurs
<code>false_discovery_</code> <code>control</code>	correction FDR	p-values ajustées

Erreurs fréquentes : données avec NaN (utiliser `nan_policy`), confondre test unilatéral et bilatéral (`alternative`), oublier que les tests supposent l'indépendance, lire la p-value comme une taille d'effet.

39.2 statsmodels

Fournit l'inférence : `sm.OLS(y, sm.add_constant(X)).fit()` (résumé complet, `conf_int`, `resid`, `get_influence`), `smf.ols("y ~ x1 + C(poste)", data=df)` (interface par formules, gère les variables catégorielles), `sm.Logit`, `statsmodels.stats.multitest.multipletests`, `statsmodels.stats.outliers_influence.variance_inflation_factor`, `statsmodels.stats.diagnostic.het_breuschpagan`, `sm.graphics.tsa.plot_acf` (autocorrélation), `mixedlm` (effets mixtes pour données groupées). Installation : `pip install statsmodels`.

39.3 Scikit-learn

Famille	Outils	Rappels
Prétraitement	StandardScaler, MinMaxScaler, RobustScaler, PowerTransformer, OneHotEncoder, SimpleImputer	fit sur l'entraînement seulement
Pipelines	Pipeline, ColumnTransformer	évite la fuite de données
Validation	train_test_split, cross_ val_score, KFold, GroupKFold, GroupShuffleSplit, TimeSeriesSplit	choisir selon la structure
Modèles	LinearRegression, Ridge, Lasso, LogisticRegression, RandomForest*, GradientBoosting*, KNeighbors*	hyperparamètres, graine
Métriques	mean_absolute_error, mean_ squared_error, r2_score, roc_auc_score, f1_score	hors échantillon
Sélection/interprétation	mutual_info_regression, permutation_importance	jamais une preuve causale
Non supervisé	PCA, KMeans, silhouette_ score	standardiser avant

Convention générale : X de forme $(n_observations, n_variables)$, y de forme $(n,)$; `fit` apprend, `predict/transform` appliquent; les attributs appris se terminent par `_`.

Outils d'analyse reproductible

- **Jupyter Notebook** : mélange code, graphiques et texte. Pièges : état caché (cellules exécutées dans le désordre); relancer régulièrement « Restart & Run All ». Séparer exploration (notebook) et fonctions réutilisables (fichiers `.py`).
- **Git et GitHub** : historique et sauvegarde du code. Réflexes : commits fréquents et descriptifs, `.gitignore` pour les gros fichiers et données, jamais de données sensibles ou de clés dans le dépôt.
- **Environnements** : `requirements.txt` ou `environment.yml` pour fixer les versions; une graine aléatoire (`random_state`, `default_rng(42)`) pour des résultats reproductibles.
- **Documentation des analyses** : pour chaque résultat, noter la question, les données et versions, les filtres appliqués, les décisions de nettoyage et leur justification, les limites. Un lecteur (ou toi dans trois mois) doit pouvoir refaire l'analyse.
- **Séparer les rôles** : données brutes (jamais modifiées), données nettoyées (produites par du code), résultats.

Attention

Les exemples numériques de ce manuel ne sont pas des sorties d'exécution recopiées : relance-les avec ta version de Python et des bibliothèques, et compare avec les valeurs théoriques calculées à la main dans le texte lorsqu'elles sont données.

Glossaire statistique

Aberrante (valeur)

Observation très éloignée des autres ; peut être une erreur ou un phénomène réel.

Alpha (α)

Seuil de décision d'un test, fixé avant l'analyse.

Autocorrélation

Corrélation d'une série avec elle-même décalée dans le temps.

Biais

Écart systématique entre une estimation et la valeur visée.

Bootstrap

Rééchantillonnage avec remise pour estimer l'incertitude.

Confusion (variable de)

Variable influençant à la fois la variable explicative et la cible.

Corrélation

Mesure d'association entre deux variables (Pearson : linéaire ; Spearman/Kendall : monotone).

Covariance

Moyenne des produits des écarts à la moyenne de deux variables.

Densité

Fonction dont l'aire représente une probabilité (variable continue).

ECDF

Fonction de répartition empirique.

Échantillon

Sous-ensemble observé d'une population.

Effet (taille d')

Mesure de l'ampleur d'un effet, indépendante de n .

Erreur-type

Écart-type d'un estimateur (par exemple $s/\sqrt{n}$ pour la moyenne).

Fuite de données

Information indisponible au moment de la prédiction utilisée à l'entraînement.

Hétéroscédasticité

Variance des erreurs non constante.

Hypothèse nulle

Hypothèse « pas d'effet » que le test cherche à mettre en défaut.

Intervalle de confiance

Plage de valeurs plausibles du paramètre, avec un niveau de confiance de procédure.

Kurtosis

Mesure de l'importance des queues d'une distribution.

MAD

Écart absolu médian, mesure de dispersion robuste.

MAR, MCAR, MNAR

Mécanismes de données manquantes.

Multicolinéarité

Forte corrélation entre variables explicatives.

Observation influente

Observation dont le retrait change sensiblement les résultats.

p-value

Probabilité, sous H_0 , d'un résultat au moins aussi extrême que celui observé.

Puissance

Probabilité de détecter un effet réel.

Quantile

Valeur sous laquelle se trouve une proportion donnée des données.

Résidu

Différence entre valeur observée et valeur prédite.

Simpson (paradoxe de)

Inversion d'une tendance quand on agrège des groupes.

Skewness

Asymétrie d'une distribution.

Variance

Moyenne des carrés des écarts à la moyenne.

Winsorisation

Remplacement des valeurs extrêmes par des percentiles choisis.

Aide-mémoire : quelle méthode choisir ?

Question	Méthode usuelle	Alternative/robuste
Deux numériques : relation linéaire	Pearson	Spearman, Kendall
Deux numériques : relation monotone	Spearman	Kendall
Dépendance quelconque	information mutuelle graphique	+ modèle non linéaire
Deux catégorielles	chi carré, V de Cramér	Fisher (petits effectifs)
Binaire vs numérique	Welch / point-bisériale	Mann–Whitney, permutation Wilcoxon
Deux mesures appariées	t apparié	Kruskal–Wallis
≥ 3 groupes	ANOVA (Welch si variances inégales)	
Post-hoc	Tukey	Mann–Whitney + Holm
Effet ajusté sur plusieurs variables	régression multiple	modèles mixtes, robustes
Normalité	Q-Q plot, Shapiro–Wilk	juger l'écart pratique
Valeurs aberrantes	IQR, MAD	modèles robustes
Incertitude sans loi	bootstrap	permutation
Prédiction	pipeline + validation par groupes	comparer à une référence

Checklist d'une EDA professionnelle

- ☐ La question et l'unité d'observation sont écrites.
- ☐ Le dictionnaire des variables est lu ; unités et définitions vérifiées.
- ☐ Dimensions et types contrôlés ; conversions faites dans le code.
- ☐ Valeurs manquantes quantifiées, mécanisme discuté, traitement documenté.
- ☐ Doublons contrôlés sur les bonnes clés.
- ☐ Clés de jointure vérifiées ; cardinalité et nombre de lignes contrôlés après chaque jointure.
- ☐ Valeurs impossibles et incohérences d'unités traitées.
- ☐ Distributions examinées (histogramme, ECDF) ; asymétrie et extrêmes notés.
- ☐ Extrêmes examinés un à un avant toute suppression ; décision justifiée.
- ☐ Structure de dépendance identifiée (joueur, match, saison, temps) ; effectifs effectifs estimés.
- ☐ Relations explorées avec plusieurs coefficients *et* graphiques.
- ☐ Sous-groupes pertinents examinés ; nombre de comparaisons recensé.
- ☐ Hypothèses écrites et séparées en confirmatoires/exploratoires.
- ☐ Tailles d'effet et intervalles rapportés, pas seulement des p-values.
- ☐ Pas de fuite de données ; validation adaptée à la structure.
- ☐ Limites et conclusions rédigées sans surinterprétation causale.
- ☐ Code reproductible (graine, versions, notebook relancé de zéro).

Références

Documentations officielles

- NumPy : <https://numpy.org/doc/>
- pandas : <https://pandas.pydata.org/docs/>
- Matplotlib : <https://matplotlib.org/stable/>
- Seaborn : <https://seaborn.pydata.org/>
- SciPy (scipy.stats) : <https://docs.scipy.org/doc/scipy/reference/stats.html>
- statsmodels : <https://www.statsmodels.org/stable/>
- Scikit-learn : <https://scikit-learn.org/stable/>
- Project Jupyter : <https://docs.jupyter.org/>
- Git : <https://git-scm.com/doc> ; GitHub : <https://docs.github.com/>

Ouvrages et sources statistiques

- NIST/SEMATECH, *e-Handbook of Statistical Methods* (en ligne, gratuit).
- L. Wasserman, *All of Statistics*, Springer.
- G. James, D. Witten, T. Hastie, R. Tibshirani, *An Introduction to Statistical Learning (with Python)*, Springer (version en ligne gratuite).
- A. Gelman, J. Hill, *Data Analysis Using Regression and Multilevel/Hierarchical Models*, Cambridge University Press.
- M. Hernán, J. Robins, *Causal Inference : What If* (version en ligne gratuite).
- B. Efron, R. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall.
- Page officielle de la compétition NFL Big Data Bowl sur Kaggle : description des données et règles à lire *en premier*.